

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN
ୱୱୱୱ



BÁO CÁO ĐỒ ÁN MÔN HỌC
ĐIỆN TOÁN ĐÁM MÂY

WEBSITE XEM THỜI TIẾT THEO
THÀNH PHỐ

GVHD: ThS. Cao Tiến Thành

SV thực hiện: Trần Phan Cảnh Phúc – 23DH112758

Chuyên ngành: An Ninh Mạng

Khóa: Khóa 29 – K2023

TP. Hồ Chí Minh, tháng 04 năm 2026

[illegible]

(Ký tên, đóng dấu)

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn thân trọng đến thầy Cao Tiên Thành - người đã tận tình hướng dẫn , chỉ bảo, giảng dạy, truyền đạt kiến thức cho em trong suốt quá trình học tập và quá trình nghiên cứu để thực hiện đề tài này .

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu, quý thầy cô trong Khoa Công Nghệ Thông Tin, Trường Đại Học Ngoại Ngữ Tin Học đã tạo điều kiện thuận lợi về cơ sở vật chất, tài liệu và môi trường học tập trong suốt quá trình thực hiện đề tài.

Xin cảm ơn bạn bè đã luôn ở bên quan tâm và giúp đỡ , hỗ trợ và tạo điều kiện thuận lợi cho nhóm chúng em được học tập, nghiên cứu , để hoàn thành đề tài.

MỤC LỤC

LỜI CẢM ƠN.....	I
MỤC LỤC	II
DANH MỤC HÌNH ẢNH.....	III
DANH MỤC BẢNG.....	IV
CHƯƠNG 1. TỔNG QUAN.....	1
1.1. Đặt vấn đề	1
1.2. Động lực nghiên cứu	1
1.3. Phạm vi đề tài	2
1.3.1. Phạm vi chức năng	2
1.3.2. Phạm vi kỹ thuật.....	2
1.3.3. Giới hạn của đề tài.....	2
1.4. Đối tượng nghiên cứu.....	3
1.4.1. Công nghệ điện toán đám mây (Cloud Computing).....	3
1.4.2. OpenWeather API	3
1.4.3. Phát triển ứng dụng Web Front-end	3
1.4.4. Quy trình triển khai lên đám mây	4
1.5. Kết quả dự kiến	4
1.5.1. Sản phẩm phần mềm.....	4
1.5.2. Kiến thức và kỹ năng đạt được	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	5
2.1. Kiến trúc ứng dụng Web cơ bản.....	5
2.1.1. Mô hình kiến trúc Client-Server	5
2.1.2. RESTful API	6
2.1.3. JSON và xử lý dữ liệu API.....	6
2.1.4. Fetch API và Async/Await trong JavaScript	6

2.2. Định nghĩa và khái niệm Cloud Computing	6
2.2.1. Định nghĩa Cloud Computing	6
2.2.2. Năm đặc điểm thiết yếu của Cloud Computing	7
2.2.3. Ba mô hình dịch vụ Cloud	7
2.2.4. PaaS (Platform as a Service) - Nền tảng được sử dụng trong đề tài	8
2.2.5. Mô hình Serverless Computing	9
2.3. So sánh các nền tảng triển khai Cloud	10
2.4. OpenWeather API	11
2.4.1. Tổng quan về OpenWeather API	11
2.4.2. Cấu trúc Request và Response	11
2.4.3. Xác thực và giới hạn API (Rate Limiting)	12
2.5. Vercel - Nền tảng triển khai	12
2.5.1. Giới thiệu về Vercel	12
2.5.2. Tính năng nổi bật của Vercel	13
2.5.3. Quy trình CI/CD trên Vercel	14
2.6. Git và GitHub trong quản lý mã nguồn	14
2.7. Supabase và cơ sở dữ liệu trong ứng dụng Web	14
2.7.1. Vai trò của cơ sở dữ liệu	15
2.7.2. Supabase	15
2.7.3. Lý do lựa chọn Supabase	16
CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN	17
3.1. Lựa chọn công nghệ và công cụ	17
3.1.1. Tổng quan stack công nghệ	17
3.1.2. Lý do lựa chọn HTML/CSS/JavaScript	17
3.2. Sơ đồ nguyên lý và kiến trúc hệ thống	17
3.2.1. Kiến trúc tổng thể	17
3.2.2. Luồng dữ liệu (Data Flow)	18

3.3. Thiết kế giao diện người dùng (UI Design)	19
3.3.1. Nguyên tắc thiết kế	19
3.3.2. Bố cục trang (Layout)	20
3.4. Cấu trúc mã nguồn	20
3.4.1. Cấu trúc thư mục dự án	20
3.4.2. Mô tả vai trò các file chính	21
3.4.3. Một số đoạn mã nguồn tiêu biểu	21
3.5. Các bước triển khai lên Vercel	24
3.5.1. Chuẩn bị môi trường phát triển trên Ubuntu/Windows	24
3.5.2. Đăng ký và lấy API Key từ OpenWeather	24
3.5.3. Đưa code lên GitHub	24
3.5.4. Triển khai lên Vercel	24
3.5.5. Bảo mật API Key với Environment Variables	25
3.6. Nâng cấp chức năng lưu vị trí với Supabase	26
3.6.1. Thiết kế bảng saved_locations	26
3.6.2. Cơ chế client_id	27
3.6.3. API lưu vị trí	27
3.6.4. API lấy danh sách và xóa vị trí đã lưu	27
3.7. Kiểm thử chức năng (Testing Plan)	28
CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ	29
4.1. Môi trường thực nghiệm	29
4.2. Kết quả triển khai giao diện ứng dụng	30
4.3. Kết quả chức năng tra cứu thời tiết	30
4.4. Kết quả chức năng sử dụng vị trí hiện tại	32
4.5. Kết quả hiển thị bản đồ vị trí	32
4.6. Kết quả chức năng dự báo thời tiết 5 ngày	33
4.7. Kết quả nâng cấp chức năng lưu vị trí với Supabase	34

4.8. Kết quả lưu dữ liệu vào Supabase	35
CHƯƠNG 5: KẾT LUẬN	36
5.1. Kết quả đạt được	36
5.2. Ưu điểm của hệ thống	36
5.3. Hạn chế của hệ thống	36
5.4. Hướng phát triển trong tương lai	37
5.5. Kết luận chung	37
TÀI LIỆU THAM KHẢO	38

DANH MỤC HÌNH ẢNH

Hình 1. OpenWeather.....	3
Hình 2. Mô hình hoạt động Client-Server	5
Hình 3. Ba mô hình dịch vụ Cloud	8
Hình 4. Dịch vụ PaaS.....	9
Hình 5. Serverless Computing	10
Hình 6. Cách Vercel hoạt động	13
Hình 7. supabase	15
Hình 8. Luồng dữ liệu hoạt động.....	19
Hình 9. Source website weather.....	20
Hình 10. Code gọi thời tiết theo vị trí thành phố.....	22
Hình 11. Code hiển thị dữ liệu lên giao diện	22
Hình 12. Code cập nhập vị trí bản đồ	23
Hình 13. Code lưu thời tiết theo thành phố	23
Hình 14. bash git đưa code lên git	24
Hình 15. bash deploy source lên vercel	25
Hình 16. Giao diện chính của ứng dụng Weather App	30
<i>Hình 17. Kết quả tra cứu thời tiết theo tên thành phố.....</i>	<i>31</i>
<i>Hình 18. Thông báo lỗi khi nhập thành phố không hợp lệ.....</i>	<i>31</i>
<i>Hình 19. Chức năng vị trí của tôi</i>	<i>32</i>
Hình 20. Bản đồ hiển thị vị trí thành phố và marker hiển thị.....	33
<i>Hình 21. Giao diện dự báo thời tiết 5 ngày</i>	<i>34</i>
Hình 22. Lưu vị trí trên giao diện	35
Hình 23. Bảng saved_locations trên Supabase	35

DANH MỤC BẢNG

Bảng 1. So sánh các nền tảng Cloud.....	10
Bảng 2. Các công nghệ phục vụ đề tài.....	17
Bảng 4. Mô tả các file của web.....	21
Bảng 5. Các trường dữ liệu chính lưu trên Supabase	26
Bảng 6. Kiểm thử các chức năng	28
Bảng 7. Dịch vụ và công cụ được sử dụng trong quá trình thực nghiệm	29

CHƯƠNG 1. TỔNG QUAN

1.1. Đặt vấn đề

Trong thời đại công nghệ số phát triển nhanh chóng hiện nay, nhu cầu tra cứu thông tin thời tiết đã trở thành một phần không thể thiếu trong cuộc sống hàng ngày của mỗi người. Từ người nông dân cần lên kế hoạch canh tác, người lao động cần chuẩn bị trang phục phù hợp, cho đến các doanh nghiệp logistics cần lên lịch vận chuyển hàng hóa, thông tin thời tiết đóng vai trò then chốt trong việc đưa ra quyết định đúng đắn và kịp thời.

Tuy nhiên, các ứng dụng tra cứu thời tiết hiện tại thường gặp một số vấn đề như: giao diện phức tạp, khó sử dụng; tốc độ tải trang chậm; không thể truy cập trên nhiều thiết bị; hoặc không hỗ trợ tra cứu thông tin thời tiết cho các thành phố cụ thể tại Việt Nam. Điều này đặt ra nhu cầu cần có một giải pháp đơn giản, hiệu quả và dễ sử dụng hơn.

Bên cạnh đó, sự phát triển mạnh mẽ của điện toán đám mây (Cloud Computing) đã mở ra nhiều cơ hội mới cho việc triển khai các ứng dụng web với chi phí thấp, khả năng mở rộng linh hoạt và tính sẵn sàng cao. Các nền tảng như Vercel cho phép các nhà phát triển triển khai ứng dụng web một cách nhanh chóng mà không cần quản lý hạ tầng máy chủ phức tạp.

Xuất phát từ thực tế đó, đề tài "Website xem thời tiết theo thành phố" được đề xuất nhằm xây dựng một ứng dụng web đơn giản, thân thiện với người dùng, cho phép tra cứu thông tin thời tiết theo thời gian thực bằng cách tích hợp API thời tiết công khai (OpenWeather API) và triển khai trên nền tảng đám mây Vercel.

1.2. Động lực nghiên cứu

Đề tài được thực hiện nhằm đáp ứng yêu cầu của môn Điện toán đám mây, đồng thời tạo cơ hội cho sinh viên áp dụng kiến thức lý thuyết vào việc xây dựng và triển khai một ứng dụng thực tế trên nền tảng cloud. Thông qua đó, người thực hiện có thể hiểu rõ hơn về quy trình phát triển và vận hành ứng dụng trong môi trường đám mây. Tiếp cận công nghệ Serverless: Vercel cung cấp môi trường triển khai serverless, giúp giảm thiểu chi phí vận hành và tối ưu hóa tài nguyên. Đây là xu hướng công nghệ đang được nhiều doanh nghiệp lớn áp dụng.

Bên cạnh đó, đề tài giúp sinh viên tìm hiểu cách tích hợp các dịch vụ bên ngoài thông qua OpenWeather API. Quá trình này góp phần nâng cao kỹ năng làm việc với REST API, xử lý dữ liệu và xây dựng các chức năng tương tác với hệ thống bên thứ ba.

Việc sử dụng nền tảng Vercel còn giúp tiếp cận mô hình Serverless hiện đại, cho phép triển khai ứng dụng nhanh chóng, tiết kiệm tài nguyên và giảm chi phí vận hành. Đây cũng là công nghệ đang được áp dụng rộng rãi trong thực tế.

1.3. Phạm vi đề tài

Để đảm bảo tính khả thi trong thời gian thực hiện đề tài, phạm vi nghiên cứu và phát triển được giới hạn trong các nội dung sau:

1.3.1. Phạm vi chức năng

Đề tài tập trung xây dựng một ứng dụng web tra cứu thời tiết cho phép người dùng tìm kiếm thông tin thời tiết hiện tại theo tên thành phố. Hệ thống cung cấp các thông tin cơ bản như nhiệt độ hiện tại, nhiệt độ cao nhất và thấp nhất, độ ẩm, tốc độ gió, mô tả tình trạng thời tiết. Bên cạnh đó, ứng dụng hiển thị biểu tượng thời tiết tương ứng với điều kiện thực tế nhằm tăng tính trực quan cho người dùng.

Ngoài thông tin thời tiết, hệ thống còn cung cấp một số dữ liệu địa lý như tên thành phố, quốc gia và múi giờ địa phương. Giao diện được thiết kế theo hướng responsive, đảm bảo khả năng sử dụng trên cả máy tính và thiết bị di động. Sau khi hoàn thành, ứng dụng được triển khai trên nền tảng Vercel và hỗ trợ truy cập thông qua tên miền công khai.

Bên cạnh đó hệ thống còn được nâng cấp thêm chức năng lưu vị trí yêu thích. Chức năng này cho phép người dùng lưu lại các địa điểm đã tra cứu, xem lại danh sách vị trí đã lưu và xóa vị trí không còn cần thiết.

1.3.2. Phạm vi kỹ thuật

- Ngôn ngữ lập trình: HTML, CSS, JavaScript
- API tích hợp: OpenWeather API
- Nền tảng triển khai: Vercel (PaaS/Serverless)
- Database: Supabase
- Hệ điều hành phát triển: Ubuntu Linux
- Công cụ quản lý phiên bản: Git, GitHub

1.3.3. Giới hạn của đề tài

Mặc dù đã đáp ứng được các yêu cầu cơ bản, hệ thống vẫn còn một số hạn chế nhất định. Hiện tại, ứng dụng chỉ hỗ trợ tra cứu thông tin thời tiết hiện tại và chưa tích hợp chức năng dự báo thời tiết trong nhiều ngày. Ngoài ra, hệ thống chưa cung cấp tính năng quản lý tài

khoản người dùng hoặc lưu lại lịch sử tra cứu. Do sử dụng gói miễn phí của OpenWeather API nên số lượng yêu cầu gửi đến dịch vụ bị giới hạn theo quy định của nhà cung cấp. Bên cạnh đó, ứng dụng chưa hỗ trợ tự động xác định vị trí hiện tại hoặc tra cứu thời tiết dựa trên tọa độ GPS của người dùng.

1.4. Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu và tìm hiểu các đối tượng chính sau:

1.4.1. Công nghệ điện toán đám mây (Cloud Computing)

Nghiên cứu các mô hình dịch vụ đám mây, đặc biệt là PaaS (Platform as a Service) và mô hình Serverless Computing. Tìm hiểu các nền tảng triển khai phổ biến hiện nay như Vercel, Netlify, và Google Cloud Platform (GCP) để lựa chọn giải pháp phù hợp nhất với yêu cầu của đề tài.

1.4.2. OpenWeather API

Nghiên cứu cách tích hợp và sử dụng OpenWeather API, bao gồm: cấu trúc request/response của API, phương thức xác thực bằng API key, các endpoint khác nhau (Current Weather, Forecast, One Call API), cách xử lý dữ liệu JSON trả về và hiển thị lên giao diện người dùng.



Hình 1. OpenWeather

1.4.3. Phát triển ứng dụng Web Front-end

Nghiên cứu các kỹ thuật xây dựng giao diện web hiện đại, responsive và thân thiện với người dùng bằng HTML, CSS và JavaScript. Áp dụng các khái niệm như Fetch API, async/await, DOM manipulation để kết nối giao diện với dữ liệu thời tiết từ API.

1.4.4. Quy trình triển khai lên đám mây

Nghiên cứu quy trình đưa ứng dụng từ môi trường phát triển lên môi trường sản xuất thông qua Vercel, bao gồm: kết nối với repository GitHub, cấu hình biến môi trường, thiết lập domain, theo dõi quá trình build và deploy tự động.

1.5. Kết quả dự kiến

Sau khi hoàn thành đề tài, nhóm kỳ vọng đạt được các kết quả sau:

1.5.1. Sản phẩm phần mềm

Kết quả mong đợi là xây dựng thành công một website tra cứu thời tiết có thể hoạt động ổn định và truy cập từ bất kỳ thiết bị nào có kết nối Internet. Ứng dụng cần có giao diện hiện đại, trực quan, dễ sử dụng và đảm bảo thời gian phản hồi nhanh. Hệ thống được tích hợp thành công với OpenWeather API để cung cấp thông tin thời tiết theo thời gian thực. Đồng thời, ứng dụng được triển khai trên nền tảng Vercel với địa chỉ truy cập công khai, đảm bảo tính sẵn sàng và thuận tiện cho người dùng. Bên cạnh đó, API key được bảo vệ an toàn thông qua cơ chế biến môi trường, góp phần nâng cao tính bảo mật của hệ thống.

1.5.2. Kiến thức và kỹ năng đạt được

Thông qua việc thực hiện đề tài, em hướng đến việc hiểu rõ kiến trúc và các mô hình dịch vụ trong điện toán đám mây, đồng thời nắm vững quy trình triển khai ứng dụng web trên các nền tảng PaaS và Serverless. Bên cạnh đó, đề tài này còn giúp em rèn luyện kỹ năng tích hợp và làm việc với các RESTful API bên ngoài, xử lý dữ liệu JSON và phát triển giao diện người dùng bằng các công nghệ front-end. Ngoài ra, quá trình triển khai và quản lý mã nguồn còn giúp em làm quen với quy trình CI/CD cơ bản thông qua việc sử dụng GitHub kết hợp với Vercel.

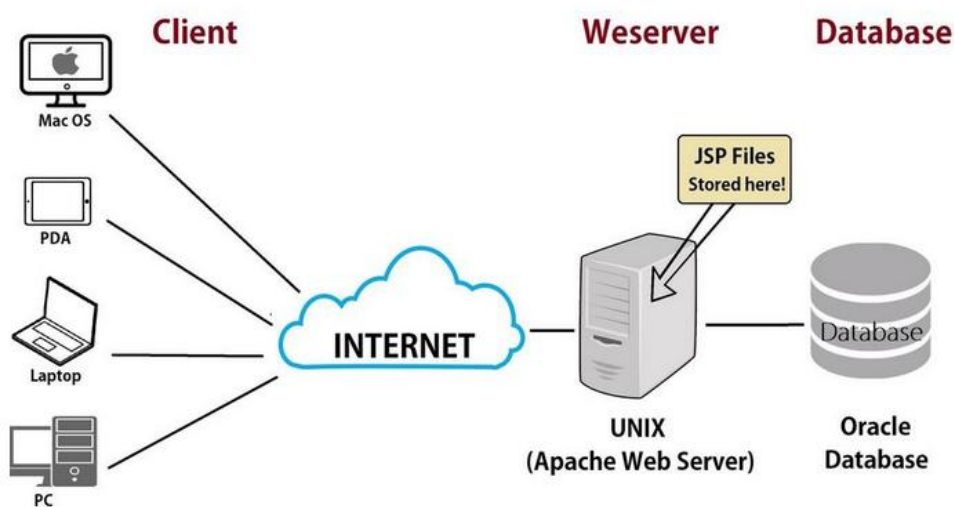
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Kiến trúc ứng dụng Web cơ bản

2.1.1. Mô hình kiến trúc Client-Server

Kiến trúc Client-Server là mô hình nền tảng của hầu hết các ứng dụng web hiện đại. Trong mô hình này, ứng dụng được chia thành hai thành phần chính:

- Client: Là phần giao diện người dùng chạy trên trình duyệt web. Client gửi yêu cầu đến server thông qua giao thức HTTP/HTTPS và hiển thị kết quả trả về cho người dùng. Trong đề tài này, client là các file HTML, CSS, JavaScript chạy trực tiếp trên trình duyệt của người dùng.
- Server: Là phần xử lý logic nghiệp vụ, quản lý dữ liệu và cung cấp các dịch vụ cho client. Server nhận yêu cầu từ client, xử lý và trả về kết quả. Trong đề tài này, Vercel đóng vai trò là server-side platform.



Hình 2. Mô hình hoạt động Client-Server

Quá trình giao tiếp giữa Client và Server diễn ra theo mô hình yêu cầu - phản hồi. Đầu tiên, Client gửi một HTTP Request đến Server để yêu cầu dữ liệu hoặc dịch vụ. Sau khi tiếp nhận yêu cầu, Server tiến hành xử lý và trả về một HTTP Response chứa kết quả tương ứng. Cuối cùng, Client nhận phản hồi và hiển thị dữ liệu cho người dùng.

2.1.2. RESTful API

REST (Representational State Transfer) là một kiểu kiến trúc phần mềm định nghĩa các ràng buộc để tạo ra các dịch vụ web. API tuân theo kiến trúc REST được gọi là RESTful API. Đây là tiêu chuẩn phổ biến nhất hiện nay để xây dựng API web.

OpenWeather API là một RESTful API điển hình. Khi gọi API, client gửi HTTP GET request kèm theo tên thành phố và API key. Server của OpenWeather xử lý và trả về dữ liệu thời tiết dưới dạng JSON (JavaScript Object Notation).

2.1.3. JSON và xử lý dữ liệu API

JSON (JavaScript Object Notation) là định dạng trao đổi dữ liệu nhẹ, dễ đọc và dễ phân tích bởi cả con người lẫn máy móc. JSON là định dạng chuẩn de facto cho việc trao đổi dữ liệu trong các RESTful API hiện đại, thay thế XML trong hầu hết các ứng dụng web.

Cấu trúc dữ liệu JSON trả về từ OpenWeather API bao gồm các trường quan trọng như: tên thành phố, tọa độ địa lý, mô tả thời tiết (weather description và icon), nhiệt độ hiện tại và cảm giác nhiệt (feels_like), độ ẩm (humidity), tốc độ gió (wind speed), áp suất khí quyển (pressure), và thời gian mặt trời mọc/lặn (sunrise/sunset).

2.1.4. Fetch API và Async/Await trong JavaScript

Fetch API là giao diện lập trình ứng dụng hiện đại được tích hợp sẵn trong trình duyệt, cho phép thực hiện các HTTP request một cách bất đồng bộ (asynchronous). Fetch API thay thế XMLHttpRequest (XHR) truyền thống với cú pháp đơn giản và sử dụng Promise.

Async/Await là cú pháp hiện đại trong JavaScript (ES2017) giúp xử lý code bất đồng bộ trở nên dễ đọc và dễ viết hơn, trông giống như code đồng bộ thông thường. Từ khóa async được đặt trước function để khai báo hàm bất đồng bộ, trong khi await được dùng để chờ kết quả của một Promise trước khi tiếp tục thực thi.

2.2. Định nghĩa và khái niệm Cloud Computing

2.2.1. Định nghĩa Cloud Computing

Điện toán đám mây (Cloud Computing) là mô hình cung cấp các tài nguyên máy tính như server, lưu trữ, cơ sở dữ liệu, mạng, phần mềm, phân tích và nhiều dịch vụ khác thông qua Internet ("đám mây") theo mô hình trả phí theo nhu cầu sử dụng (pay-as-you-go).

Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) định nghĩa điện toán đám mây là mô hình cho phép truy cập mạng thuận tiện, theo yêu cầu đến một tập hợp tài nguyên điện

toán có thể cấu hình được (ví dụ: mạng, máy chủ, lưu trữ, ứng dụng và dịch vụ) có thể được cấp phát và giải phóng nhanh chóng với nỗ lực quản lý tối thiểu hoặc tương tác tối thiểu với nhà cung cấp dịch vụ.

2.2.2. Năm đặc điểm thiết yếu của Cloud Computing

Theo NIST, điện toán đám mây có năm đặc điểm thiết yếu:

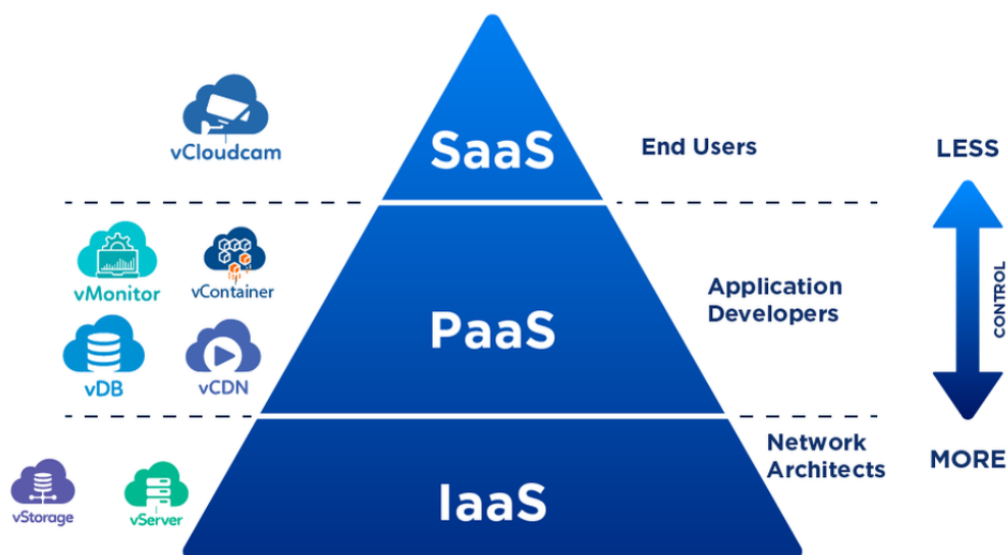
1. On-demand Self-service (Tự phục vụ theo yêu cầu): Người dùng có thể tự cấp phát tài nguyên máy tính như thời gian máy chủ và lưu trữ mạng theo nhu cầu một cách tự động, không cần tương tác với con người tại phía nhà cung cấp dịch vụ.
2. Broad Network Access (Truy cập mạng rộng): Các khả năng có sẵn qua mạng và được truy cập thông qua các cơ chế tiêu chuẩn, thúc đẩy việc sử dụng bởi các nền tảng client đa dạng như điện thoại di động, tablet, laptop và máy trạm.
3. Resource Pooling (Gộp tài nguyên): Tài nguyên máy tính của nhà cung cấp được gộp lại để phục vụ nhiều người dùng sử dụng mô hình multi-tenant, với các tài nguyên vật lý và ảo khác nhau được gán và tái gán động theo nhu cầu của người dùng.
4. Rapid Elasticity (Tính co giãn nhanh): Các khả năng có thể được cung cấp và giải phóng một cách linh hoạt, trong một số trường hợp là tự động, để mở rộng nhanh chóng theo nhu cầu.
5. Measured Service (Dịch vụ đo lường được): Hệ thống đám mây tự động kiểm soát và tối ưu hóa việc sử dụng tài nguyên bằng cách tận dụng khả năng đo lường ở một mức độ trừu tượng nào đó phù hợp với loại dịch vụ.

2.2.3. Ba mô hình dịch vụ Cloud

Điện toán đám mây cung cấp ba mô hình dịch vụ chính, thường được biết đến với tên gọi "Cloud Service Models":

- IaaS (Infrastructure as a Service - Hạ tầng như dịch vụ): Cung cấp tài nguyên máy tính cơ bản như máy chủ ảo, lưu trữ, mạng. Người dùng quản lý hệ điều hành, middleware và ứng dụng. Ví dụ điển hình: Amazon EC2, Google Compute Engine, Microsoft Azure Virtual Machines.
- PaaS (Platform as a Service - Nền tảng như dịch vụ): Cung cấp môi trường phát triển và triển khai ứng dụng. Nhà cung cấp quản lý hạ tầng bên dưới, người dùng chỉ cần tập trung vào phát triển ứng dụng. Đây là mô hình mà Vercel và Netlify áp dụng. Ví dụ: Google App Engine, Heroku, Vercel, Netlify.

- SaaS (Software as a Service - Phần mềm như dịch vụ): Cung cấp phần mềm hoàn chỉnh qua Internet. Người dùng không cần cài đặt hay quản lý bất kỳ thứ gì. Ví dụ: Gmail, Google Docs, Salesforce, Microsoft 365.



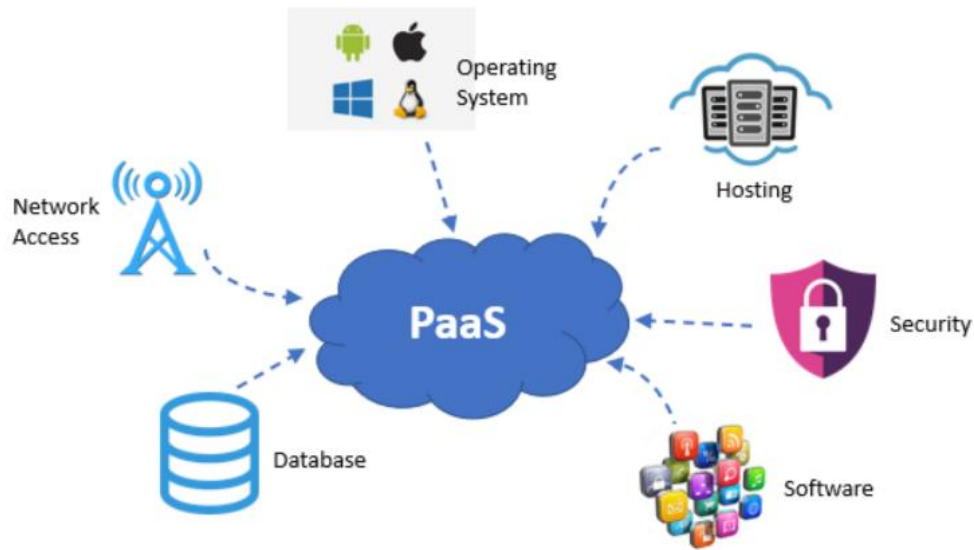
Hình 3. Ba mô hình dịch vụ Cloud

2.2.4. PaaS (Platform as a Service) - Nền tảng được sử dụng trong đề tài

PaaS (Platform as a Service) là mô hình dịch vụ đám mây cung cấp nền tảng cho phép các nhà phát triển xây dựng, chạy và quản lý ứng dụng mà không cần phải quan tâm đến việc xây dựng và duy trì cơ sở hạ tầng thường gắn liền với quá trình phát triển và khởi chạy ứng dụng.

Với PaaS, nhà cung cấp dịch vụ đám mây quản lý: phần cứng máy chủ, hệ điều hành, cơ sở hạ tầng mạng, cập nhật bảo mật, và các middleware. Trong khi đó, người dùng (developer) chỉ cần tập trung vào: code ứng dụng, dữ liệu và logic nghiệp vụ.

Đặc điểm nổi bật của PaaS trong đề tài này là Vercel cung cấp môi trường triển khai serverless, nghĩa là không cần cấu hình hay quản lý máy chủ. Developer chỉ cần push code lên GitHub, Vercel tự động build và deploy ứng dụng.



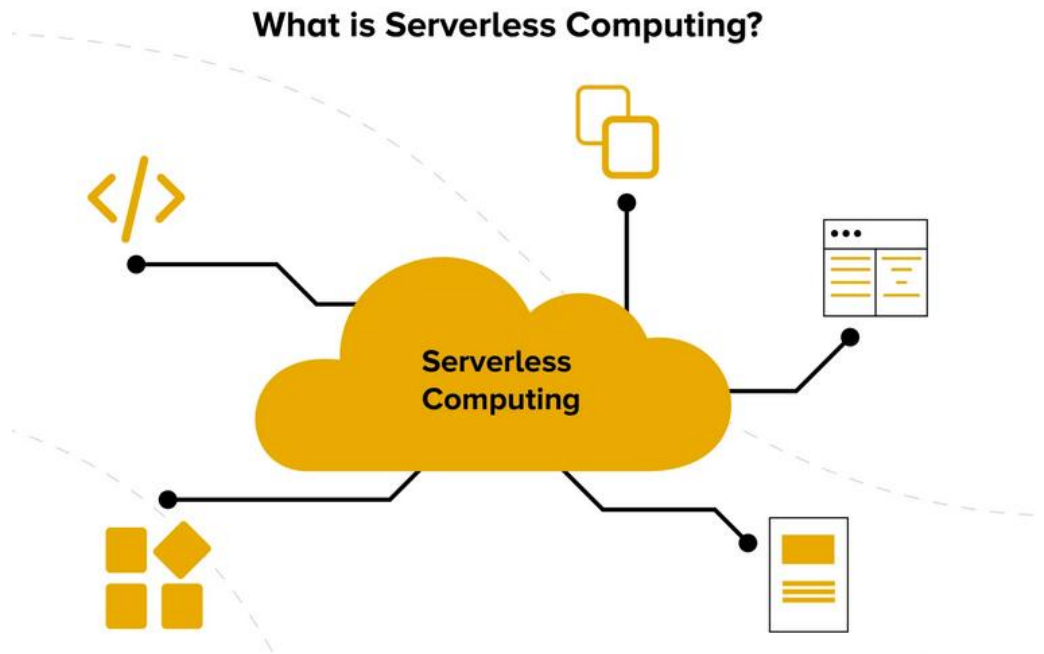
Hình 4. Dịch vụ PaaS

2.2.5. Mô hình Serverless Computing

Serverless Computing (hay còn gọi là Function as a Service - FaaS) là mô hình thực thi cloud trong đó nhà cung cấp đám mây tự động quản lý việc cấp phát và giải phóng server. Tên gọi "serverless" không có nghĩa là không có server, mà là developer không cần phải quan tâm đến việc quản lý server.

Ưu điểm của Serverless Computing bao gồm: không cần quản lý máy chủ, chi phí thấp (chỉ trả tiền khi có request), tự động mở rộng (auto-scaling) theo tải lượng, độ sẵn sàng cao (high availability), và thời gian triển khai nhanh chóng.

Vercel áp dụng mô hình Serverless Functions, cho phép các ứng dụng web tĩnh tích hợp các API endpoint được xử lý theo từng request mà không cần duy trì server chạy liên tục.



Hình 5. Serverless Computing

2.3. So sánh các nền tảng triển khai Cloud

Để lựa chọn nền tảng triển khai phù hợp, nhóm đã tiến hành nghiên cứu và so sánh các nền tảng phổ biến dành cho ứng dụng web tĩnh và JAMstack:

Bảng 1. So sánh các nền tảng Cloud

Tiêu chí	Vercel	Netlify	GCP (Firebase Hosting)
Dễ sử dụng	Rất cao - UI đơn giản, tích hợp GitHub 1 click	Cao - tương tự Vercel	Trung bình - cần CLI hoặc Console
Gói miễn phí	Rộng rãi (100GB bandwidth/tháng)	Rộng rãi (100GB bandwidth/tháng)	Có (nhưng giới hạn hơn)
Serverless Functions	Có (Vercel Functions)	Có (Netlify Functions)	Có (Cloud Functions)
Auto Deploy từ Git	Có (GitHub, GitLab, Bitbucket)	Có (GitHub, GitLab, Bitbucket)	Có (qua Cloud Build)

CDN toàn cầu	Có (Edge Network)	Có (ADN)	Có (Google CDN)
Custom Domain	Miễn phí, hỗ trợ SSL	Miễn phí, hỗ trợ SSL	Có, hỗ trợ SSL
Preview Deployments	Có (mỗi PR tạo URL preview)	Có	Không có sẵn
Tốc độ deploy	Rất nhanh (30-60 giây)	Nhanh (1-2 phút)	Trung bình (2-5 phút)
Phù hợp đề tài	Tốt nhất	Tốt	Phức tạp hơn cần thiết

Qua bảng so sánh trên, Vercel được lựa chọn vì đáp ứng tốt nhất các tiêu chí: giao diện đơn giản, tích hợp GitHub dễ dàng, gói miễn phí rộng rãi phù hợp với dự án học tập, và tốc độ deploy nhanh nhất trong các nền tảng được xem xét.

2.4. OpenWeather API

2.4.1. Tổng quan về OpenWeather API

OpenWeatherMap là nền tảng cung cấp dữ liệu thời tiết trực tuyến, được thành lập năm 2012, hiện phục vụ hơn 3 triệu nhà phát triển và 300.000 khách hàng doanh nghiệp trên toàn cầu. OpenWeather thu thập dữ liệu thời tiết từ hơn 40.000 trạm thời tiết, vệ tinh khí tượng, radar và các mô hình dự báo số trị toàn cầu (NWP models).

OpenWeather API cung cấp nhiều loại endpoint phục vụ các mục đích khác nhau. Đối với đề tài này, endpoint được sử dụng chính là Current Weather Data API (phiên bản 2.5), cho phép truy vấn dữ liệu thời tiết hiện tại theo tên thành phố, tọa độ địa lý, hoặc mã zip code.

2.4.2. Cấu trúc Request và Response

URL endpoint cơ bản của Current Weather API có cấu trúc:

`https://api.openweathermap.org/data/2.5/weather?q={city_name}&appid={API_key}&units=metric&lang=vi`

Trong đó các tham số bao gồm: q là tên thành phố cần tra cứu; appid là API key xác thực người dùng; units=metric để nhận nhiệt độ theo độ Celsius; lang=vi để nhận mô tả thời tiết bằng tiếng Việt.

Response trả về là một đối tượng JSON chứa đầy đủ thông tin thời tiết bao gồm: thông tin thành phố (tên, quốc gia, tọa độ), dữ liệu thời tiết chính (mô tả, icon), nhiệt độ (temp, feels_like, temp_min, temp_max), các chỉ số khí tượng (pressure, humidity, visibility), tốc độ và hướng gió (speed, deg), và thông tin mặt trời (sunrise, sunset).

2.4.3. Xác thực và giới hạn API (Rate Limiting)

OpenWeather API sử dụng cơ chế xác thực bằng API Key. Mỗi nhà phát triển cần đăng ký tài khoản tại openweathermap.org và tạo API key miễn phí. API key phải được gửi kèm trong mỗi request thông qua tham số appid trong URL hoặc header.

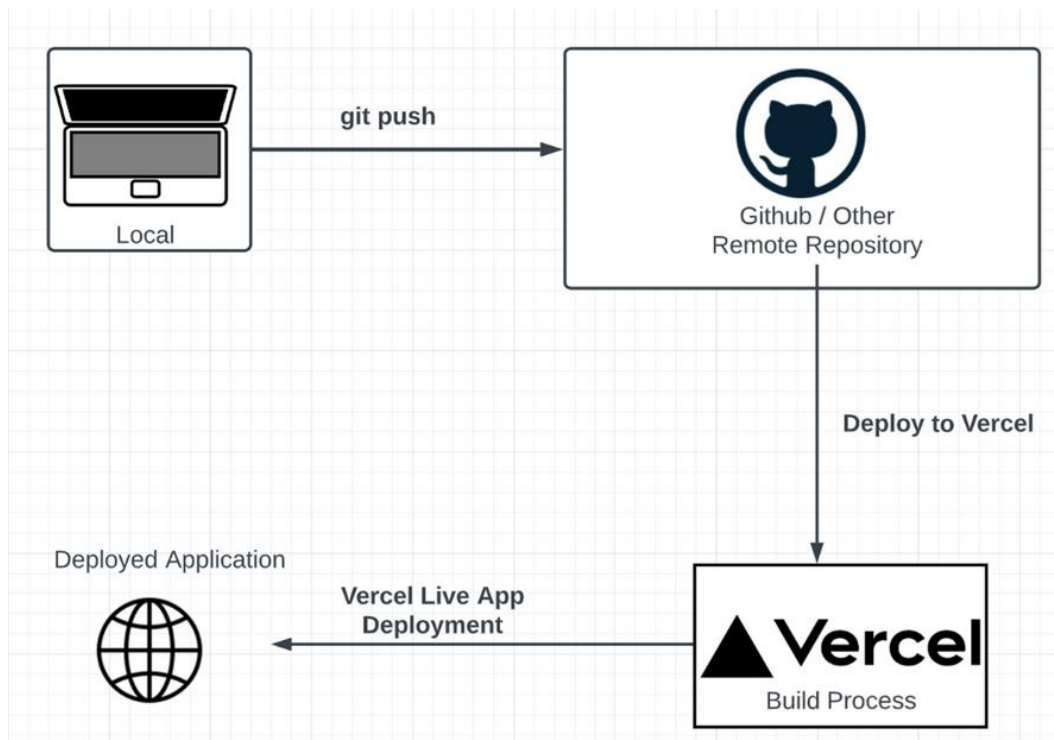
Gói miễn phí (Free tier) của OpenWeather API có các giới hạn sau: 1.000 lượt gọi API mỗi ngày, 60 lượt gọi mỗi phút (rate limit), dữ liệu cập nhật mỗi 10 phút, và chỉ bao gồm dữ liệu thời tiết hiện tại và dự báo cơ bản. Để bảo mật, API key không được nhúng trực tiếp vào code front-end mà phải được lưu trữ dưới dạng biến môi trường (Environment Variable) trên Vercel.

2.5. Vercel - Nền tảng triển khai

2.5.1. Giới thiệu về Vercel

Vercel (trước đây có tên là ZEIT) là công ty và nền tảng đám mây được thành lập năm 2015 bởi Guillermo Rauch, tập trung vào việc triển khai và lưu trữ các ứng dụng web tĩnh và JAMstack (JavaScript, APIs, Markup). Vercel là công ty đứng sau framework Next.js - một trong những framework React phổ biến nhất hiện nay.

Vercel cung cấp mô hình triển khai theo kiểu Serverless và Edge Computing, cho phép các ứng dụng được phục vụ từ các máy chủ gần nhất với người dùng cuối, giảm thiểu độ trễ mạng (network latency) và cải thiện trải nghiệm người dùng đáng kể.



Hình 6. Cách Vercel hoạt động

2.5.2. Tính năng nổi bật của Vercel

Vercel cung cấp tính năng tích hợp Git, cho phép tự động triển khai ứng dụng mỗi khi có mã nguồn mới được đẩy lên các nhánh được cấu hình như main hoặc master. Nền tảng hỗ trợ kết nối trực tiếp với GitHub, GitLab và Bitbucket, giúp đơn giản hóa quy trình triển khai.

Một tính năng nổi bật khác là Preview Deployments. Mỗi Pull Request sẽ tự động tạo một đường dẫn xem trước, cho phép kiểm tra và đánh giá các thay đổi trước khi hợp nhất vào nhánh chính, góp phần nâng cao chất lượng sản phẩm.

Vercel cũng hỗ trợ quản lý biến môi trường một cách an toàn. Người dùng có thể cấu hình riêng cho các môi trường Development, Preview và Production, giúp bảo vệ thông tin nhạy cảm như API key và các thông số cấu hình hệ thống.

Ngoài ra, Vercel sử dụng mạng phân phối nội dung toàn cầu, cho phép lưu trữ bộ nhớ đệm và phân phối nội dung từ nhiều vị trí trên thế giới. Điều này giúp cải thiện tốc độ tải trang và nâng cao trải nghiệm người dùng.

Bên cạnh việc triển khai ứng dụng tĩnh, Vercel còn hỗ trợ Vercel Functions, cho phép xây dựng và chạy các hàm phía máy chủ mà không cần quản lý hạ tầng server. Nền tảng hỗ trợ nhiều ngôn ngữ lập trình như Node.js, Python, Go và Ruby.

Cuối cùng, Vercel cung cấp công cụ Analytics để theo dõi hiệu suất website và các chỉ số Core Web Vitals. Đồng thời, nền tảng hỗ trợ kết nối tên miền tùy chỉnh cùng chứng chỉ SSL tự động, giúp tăng cường bảo mật và tính chuyên nghiệp cho ứng dụng.

2.5.3. Quy trình CI/CD trên Vercel

CI/CD (Continuous Integration/Continuous Deployment) là quy trình tự động hóa việc kiểm tra và triển khai code. Với Vercel, quy trình này diễn ra tự động:

- Developer viết code và commit lên GitHub repository
- Vercel phát hiện commit mới thông qua GitHub Webhook
- Vercel tự động build ứng dụng (nếu có bước build)
- Sau khi build thành công, Vercel deploy lên môi trường Production hoặc Preview
- URL deployment mới được tạo và có thể truy cập ngay lập tức

Toàn bộ quy trình này diễn ra trong vòng 30-60 giây, cho phép developer tập trung hoàn toàn vào việc viết code mà không cần lo lắng về quá trình triển khai.

2.6. Git và GitHub trong quản lý mã nguồn

Git là hệ thống quản lý phiên bản phân tán (Distributed Version Control System - DVCS) được phát triển bởi Linus Torvalds năm 2005. Git cho phép nhiều người cùng làm việc trên một dự án mà không xung đột, theo dõi lịch sử thay đổi của code, và dễ dàng khôi phục về các phiên bản trước.

GitHub là nền tảng lưu trữ code trực tuyến sử dụng Git, cung cấp giao diện web thân thiện và các tính năng cộng tác như Pull Requests, Issues, Actions (CI/CD). Trong đề tài này, GitHub được sử dụng để lưu trữ mã nguồn và là cầu nối để Vercel tự động triển khai ứng dụng.

2.7. Supabase và cơ sở dữ liệu trong ứng dụng Web

Trong ứng dụng Web, cơ sở dữ liệu được sử dụng để lưu trữ và quản lý dữ liệu phát sinh trong quá trình người dùng sử dụng hệ thống. Đối với đề tài này, các chức năng chính như tra cứu thời tiết, hiển thị bản đồ và dự báo thời tiết có thể hoạt động thông qua OpenWeather API. Tuy nhiên, khi ứng dụng được nâng cấp thêm chức năng lưu vị trí yêu thích, hệ thống cần có cơ sở dữ liệu để lưu lại các địa điểm mà người dùng đã chọn.

Supabase được sử dụng trong đề tài như một nền tảng cơ sở dữ liệu trên Cloud. Dữ liệu vị trí đã lưu được lưu trong bảng `saved_locations`, bao gồm các thông tin như tên thành phố, quốc gia, tọa độ, nhiệt độ, độ ẩm và thời gian cập nhật. Việc sử dụng Supabase giúp ứng dụng có khả năng lưu trữ dữ liệu thực tế, thay vì chỉ hiển thị thông tin thời tiết tạm thời.



Hình 7. supabase

2.7.1. Vai trò của cơ sở dữ liệu

Cơ sở dữ liệu có vai trò lưu trữ, tổ chức và quản lý dữ liệu của hệ thống. Trong đề tài này, cơ sở dữ liệu được dùng để lưu danh sách vị trí yêu thích của người dùng. Khi người dùng bấm nút “Lưu vị trí”, dữ liệu vị trí hiện tại sẽ được gửi đến Serverless Function trên Vercel, sau đó được lưu vào Supabase.

Nhờ có cơ sở dữ liệu, ứng dụng có thể hiển thị lại danh sách các vị trí đã lưu, cho phép người dùng xem lại hoặc xóa vị trí khi không còn cần thiết. Điều này giúp ứng dụng có tính thực tế hơn so với việc chỉ tra cứu thời tiết đơn thuần.

2.7.2. Supabase

Supabase là một nền tảng Backend-as-a-Service, cung cấp cơ sở dữ liệu PostgreSQL trên nền tảng đám mây. Supabase cho phép nhà phát triển tạo bảng, lưu trữ dữ liệu và quản lý dữ liệu thông qua giao diện trực quan.

Trong đề tài, Supabase được tích hợp với Vercel Serverless Functions để xử lý các thao tác lưu, lấy danh sách và xóa vị trí đã lưu. Front-end không kết nối trực tiếp đến Supabase mà gửi yêu cầu đến các API nội bộ như `/api/save-location` và `/api/saved-locations`. Cách làm này giúp hệ thống rõ ràng hơn và bảo vệ thông tin kết nối cơ sở dữ liệu.

2.7.3. Lý do lựa chọn Supabase

Supabase được lựa chọn vì dễ sử dụng, phù hợp với ứng dụng Web triển khai trên Cloud và tích hợp tốt với JavaScript. Nhóm không cần tự cài đặt máy chủ cơ sở dữ liệu riêng mà vẫn có thể tạo bảng, lưu dữ liệu và kiểm tra kết quả trực tiếp trên giao diện Supabase.

Ngoài ra, Supabase sử dụng PostgreSQL nên dữ liệu được tổ chức rõ ràng theo dạng bảng. Điều này phù hợp với chức năng lưu vị trí yêu thích của ứng dụng. Việc tích hợp Supabase giúp đề tài có thêm phần nâng cấp thực tế, thể hiện được luồng hoạt động giữa Front-end, Serverless API và Database.

CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

3.1. Lựa chọn công nghệ và công cụ

3.1.1. Tổng quan stack công nghệ

Dựa trên yêu cầu của đề tài và phân tích trong chương 2, nhóm lựa chọn stack công nghệ sau:

Bảng 2. Các công nghệ phục vụ đề tài

Thành phần	Công nghệ/Công cụ	Mục đích
Front-end	HTML, CSS, JavaScript	Xây dựng giao diện người dùng
API Data	OpenWeather API v2.5	Lấy dữ liệu thời tiết thời gian thực
Cloud Platform	Vercel (PaaS/Serverless)	Lưu trữ và triển khai ứng dụng
Version Control	Git + GitHub	Quản lý mã nguồn, CI/CD
Development OS	Ubuntu 22.04 LTS	Môi trường phát triển
Package Manager	npm	Quản lý dependencies (nếu cần)

3.1.2. Lý do lựa chọn HTML/CSS/JavaScript

Thay vì sử dụng các framework như React, Vue hay Angular, đề tài lựa chọn HTML/CSS/JavaScript vì các lý do sau:

- Đơn giản và nhanh: Không cần build process phức tạp, code chạy trực tiếp trên trình duyệt
- Phù hợp phạm vi đề tài: Ứng dụng đơn giản không cần state management phức tạp
- Hiệu suất cao: Không có overhead từ framework, load time nhanh hơn
- Dễ triển khai: File tĩnh có thể host trực tiếp trên Vercel mà không cần cấu hình build
- Tập trung vào Cloud: Mục tiêu chính là học cách triển khai trên cloud, không phải framework

3.2. Sơ đồ nguyên lý và kiến trúc hệ thống

3.2.1. Kiến trúc tổng thể

Kiến trúc tổng thể của hệ thống được xây dựng theo mô hình ứng dụng Web triển khai trên nền tảng đám mây. Hệ thống bao gồm 4 lớp chính: tầng giao diện người dùng, tầng xử lý phía server, tầng dịch vụ API bên ngoài và tầng cơ sở dữ liệu.

Lớp 1 - Presentation Layer (Tầng trình bày):

Đây là giao diện Web chạy trên trình duyệt của người dùng, được xây dựng bằng HTML, CSS và JavaScript. Tầng này chịu trách nhiệm hiển thị giao diện, nhận thao tác từ người dùng, hiển thị thông tin thời tiết, bản đồ, dự báo 5 ngày và danh sách vị trí đã lưu.

Lớp 2 - Serverless Function Layer (Tầng xử lý phía server):

Tầng này được triển khai bằng các Serverless Functions trên Vercel, nằm trong thư mục api/. Các API như /api/weather, /api/forecast, /api/save-location và /api/saved-locations có nhiệm vụ xử lý request từ Front-end, gọi dịch vụ bên ngoài hoặc kết nối đến cơ sở dữ liệu. Việc sử dụng Serverless Functions giúp che giấu API Key và không để Front-end gọi trực tiếp đến các dịch vụ quan trọng.

Lớp 3 - External API Layer (Tầng API bên ngoài):

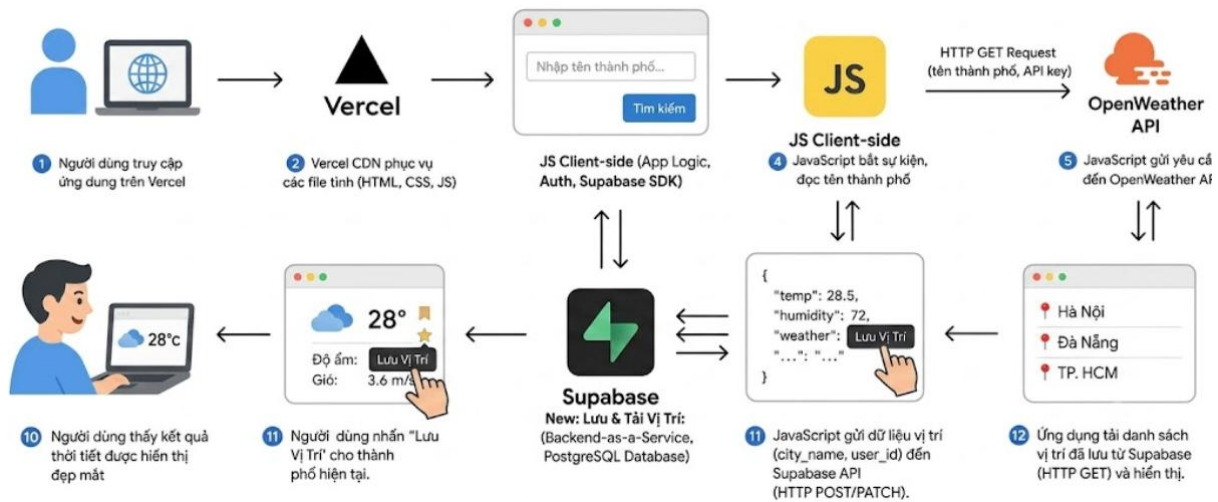
OpenWeather API được sử dụng để cung cấp dữ liệu thời tiết hiện tại và dữ liệu dự báo thời tiết. Dữ liệu trả về có định dạng JSON, sau đó được Serverless Function xử lý và gửi về Front-end để hiển thị cho người dùng.

Lớp 4 - Database Layer (Tầng cơ sở dữ liệu):

Supabase được sử dụng làm cơ sở dữ liệu để lưu trữ danh sách vị trí yêu thích của người dùng. Các dữ liệu như tên thành phố, quốc gia, tọa độ, nhiệt độ, độ ẩm và thời gian cập nhật được lưu trong bảng saved_locations. Tầng cơ sở dữ liệu không được truy cập trực tiếp từ trình duyệt mà thông qua các API trung gian trên Vercel.

3.2.2. Luồng dữ liệu (Data Flow)

Luồng hoạt động của ứng dụng khi người dùng tra cứu thời tiết diễn ra theo các bước sau



Hình 8. Luồng dữ liệu hoạt động

Khi người dùng truy cập ứng dụng trên Vercel, các tệp tĩnh được tải từ CDN để hiển thị giao diện. Ngay lúc này, JavaScript sẽ đồng thời gửi yêu cầu đến Supabase để tải danh sách các vị trí/thành phố yêu thích mà người dùng đã lưu trước đó và hiển thị lên màn hình.

Tiếp theo, khi người dùng nhập tên thành phố (hoặc chọn từ danh sách đã lưu) và nhấn tìm kiếm, JavaScript sẽ gửi HTTP Request đến OpenWeather API để lấy dữ liệu thời tiết. Máy chủ OpenWeather xử lý yêu cầu và trả về kết quả dưới dạng JSON. JavaScript phân tích dữ liệu, cập nhật giao diện để hiển thị thông tin thời tiết trực quan cho người dùng.

Đặc biệt, nếu người dùng muốn lưu lại vị trí này, họ có thể nhấn nút "Lưu vị trí". Khi đó, JavaScript sẽ lập tức gửi dữ liệu vị trí vừa chọn (như tên thành phố, ID người dùng) thông qua HTTP POST/PATCH tới Supabase API để lưu trữ an toàn vào cơ sở dữ liệu PostgreSQL cho các lần truy cập sau.

3.3. Thiết kế giao diện người dùng (UI Design)

3.3.1. Nguyên tắc thiết kế

Giao diện ứng dụng được thiết kế theo phong cách tối giản, chỉ hiển thị các thông tin cần thiết nhằm giúp người dùng dễ dàng tìm kiếm và tiếp cận dữ liệu thời tiết. Bên cạnh đó, nguyên tắc phân cấp trực quan được áp dụng để làm nổi bật các thông tin quan trọng như nhiệt độ và tình trạng thời tiết hiện tại. Về mặt thẩm mỹ, ứng dụng sử dụng tông màu xanh lam kết hợp hiệu ứng gradient, tạo cảm giác liên tưởng đến bầu trời và phù hợp với chủ đề thời tiết. Ngoài ra, các yếu tố về khả năng tiếp cận cũng được chú trọng thông qua việc sử dụng độ tương phản màu sắc hợp lý và kích thước chữ phù hợp, giúp người dùng dễ dàng đọc và sử dụng trên nhiều loại thiết bị khác nhau.

3.3.2. Bố cục trang (Layout)

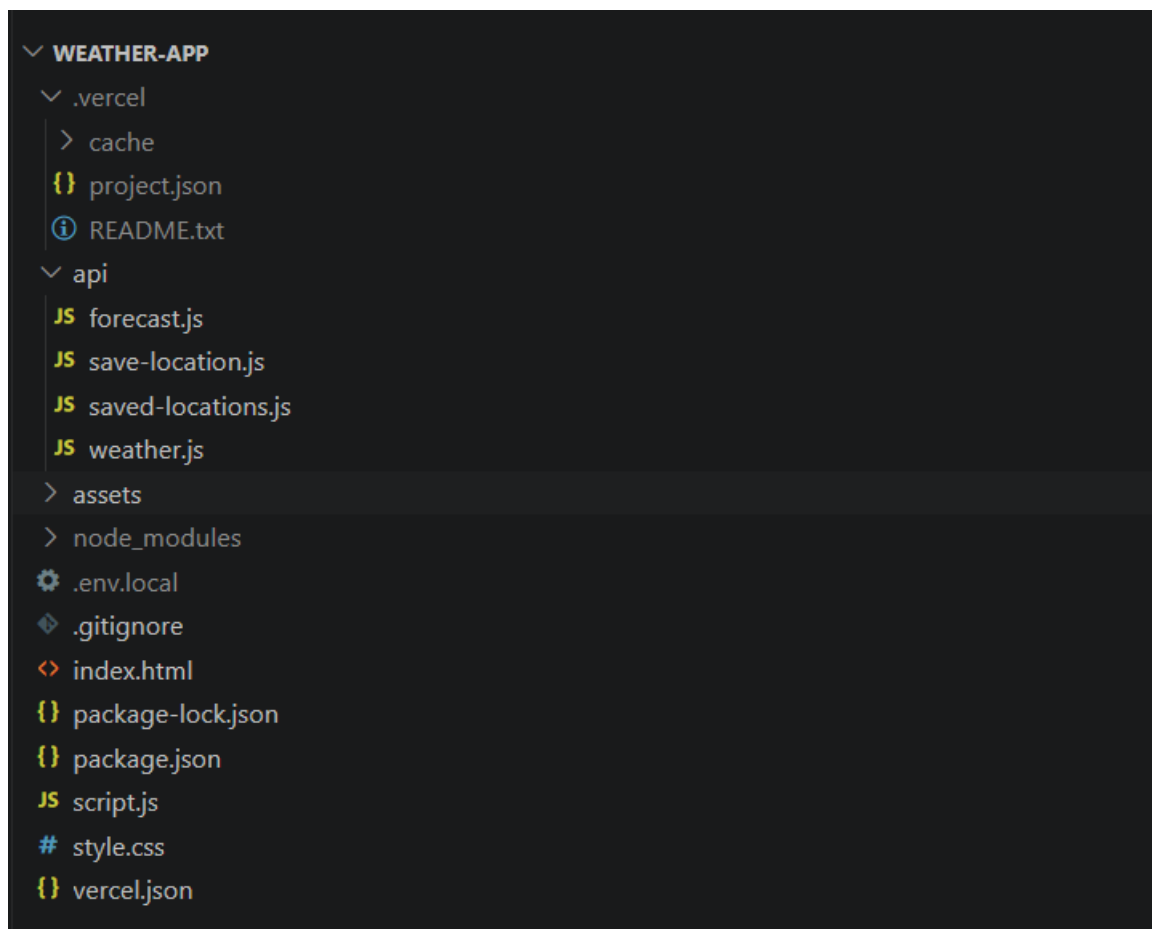
Trang web được thiết kế với bố cục một cột (single column) đơn giản, gồm các phần chính:

- Header: Tiêu đề ứng dụng "Weather App" với biểu tượng đám mây
- Search Section: Ô nhập tên thành phố và nút tìm kiếm
- Weather Card: Thẻ hiển thị thông tin thời tiết sau khi tìm kiếm thành công, bao gồm: tên thành phố và quốc gia, ngày giờ hiện tại, biểu tượng thời tiết (weather icon từ OpenWeather), nhiệt độ hiện tại (nổi bật nhất, font lớn), mô tả thời tiết (sunny, cloudy...), và các chỉ số chi tiết (độ ẩm, tốc độ gió, cảm giác nhiệt, áp suất)
- Error Message: Thông báo lỗi khi thành phố không tìm thấy hoặc có lỗi kết nối
- Loading Indicator: Hiệu ứng loading khi đang chờ kết quả từ API

3.4. Cấu trúc mã nguồn

3.4.1. Cấu trúc thư mục dự án

Mã nguồn được tổ chức theo cấu trúc thư mục đơn giản, phù hợp với ứng dụng tĩnh triển khai trên Vercel:



Hình 9. Source website weather

3.4.2. Mô tả vai trò các file chính

Mã nguồn của ứng dụng được tổ chức theo từng file riêng để dễ quản lý, bảo trì và mở rộng chức năng. Mỗi file đảm nhiệm một vai trò cụ thể trong hệ thống.

Bảng 3. Mô tả các file của web

File/Thư mục	Vai trò
index.html	Xây dựng cấu trúc giao diện chính của ứng dụng.
style.css	Thiết kế giao diện, màu sắc, bố cục và responsive.
script.js	Xử lý logic Front-end như tìm kiếm thời tiết, hiển thị dữ liệu, bản đồ, dự báo và lưu vị trí.
api/weather.js	Lấy dữ liệu thời tiết hiện tại từ OpenWeather API.
api/forecast.js	Lấy và xử lý dữ liệu dự báo thời tiết 5 ngày.
api/save-location.js	Lưu vị trí yêu thích của người dùng vào Supabase.
api/saved-locations.js	Lấy danh sách và xóa các vị trí đã lưu.
assets/	Chứa tài nguyên tĩnh như favicon hoặc icon.
package.json	Khai báo thông tin project và các thư viện sử dụng.
.env.local	Lưu các biến môi trường khi chạy local, không đưa lên GitHub.
.gitignore	Khai báo các file không đưa lên GitHub như .env.local, node_modules.

3.4.3. Một số đoạn mã nguồn tiêu biểu

- Mã nguồn gọi API thời tiết theo tên thành phố

```

async function getWeatherByCity(city) {
  showLoading();
  hideError();
  hideWeatherCard();

  try {
    const response = await fetch(`/api/weather?city=${encodeURIComponent(city)}`);
    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.message || "Không tìm thấy dữ liệu thời tiết.");
    }

    localStorage.setItem("lastCity", city);
    displayWeather(data);
  } catch (error) {
    showError(error.message);
  } finally {
    hideLoading();
  }
}

```

Hình 10. Code gọi thời tiết theo vị trí thành phố

- Mã nguồn hiển thị dữ liệu thời tiết lên giao diện

```

function displayWeather(data) {
  currentWeatherData = data;

  const cityName = data.name || "Không xác định";
  const country = data.sys?.country || "";
  const weather = data.weather?.[0];
  const coord = data.coord;

  document.getElementById("location").textContent = country
    ? `${cityName}, ${country}`
    : cityName;

  document.getElementById("dateTime").textContent = formatCityDate(
    data.dt,
    data.timezone
  );

  document.getElementById("temperature").textContent =
    `${Math.round(data.main.temp)}°C`;

  document.getElementById("description").textContent =
    capitalizeFirstLetter(weather?.description || "Không có mô tả");

  document.getElementById("humidity").textContent = `${data.main.humidity}%`;
  document.getElementById("wind").textContent =
    `${Math.round(data.wind.speed * 3.6)} km/h`;

  document.getElementById("feelsLike").textContent =
    `${Math.round(data.main.feels_like)}°C`;

  document.getElementById("pressure").textContent =
    `${data.main.pressure} hPa`;
}

```

Hình 11. Code hiển thị dữ liệu lên giao diện

- Mã nguồn cập nhật bản đồ vị trí

```
function updateMap(lat, lon, cityName, country, temp) {
  document.getElementById("mapCoords").textContent =
    `${lat.toFixed(4)}, ${lon.toFixed(4)}`;

  if (!map) {
    map = L.map("map").setView([lat, lon], 11);

    L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
      maxZoom: 19,
      attribution: "© OpenStreetMap"
    }).addTo(map);
  } else {
    map.setView([lat, lon], 11);
  }

  const temperatureIcon = L.divIcon({
    className: "temperature-marker-wrapper",
    html: `
      <div class="temperature-marker">
        🌡️ ${temp}°C
      </div>
    `,
    iconSize: [90, 42],
    iconAnchor: [45, 42],
    popupAnchor: [0, -42]
  });
}
```

Hình 12. Code cập nhật vị trí bản đồ

- Mã nguồn lưu vị trí yêu thích

```
async function saveCurrentLocation() {
  if (!currentWeatherData) {
    showSaveStatus("Chưa có vị trí để lưu. Vui lòng tìm kiếm trước.", true);
    return;
  }

  const coord = currentWeatherData.coord;

  if (!coord || typeof coord.lat !== "number" || typeof coord.lon !== "number") {
    showSaveStatus("Dữ liệu tọa độ không hợp lệ.", true);
    return;
  }

  const city = currentWeatherData.name || "Không xác định";
  const country = currentWeatherData.sys?.country || "";
  const displayAddress = country ? `${city}, ${country}` : city;

  const payload = {
    client_id: getClientId(),
    city,
    country,
    display_address: displayAddress,
    latitude: coord.lat,
    longitude: coord.lon,
    temperature: currentWeatherData.main?.temp,
    humidity: currentWeatherData.main?.humidity
  };
}
```

Hình 13. Code lưu thời tiết theo thành phố

3.5. Các bước triển khai lên Vercel

3.5.1. Chuẩn bị môi trường phát triển trên Ubuntu/Windows

Để triển khai ứng dụng, trước tiên cần chuẩn bị môi trường phát triển trên Ubuntu. Các công cụ cần cài đặt bao gồm Git để quản lý mã nguồn, Node.js và npm để hỗ trợ các gói JavaScript, cùng với Vercel CLI để thực hiện triển khai ứng dụng lên nền tảng Vercel. Sau khi cài đặt hoàn tất, cần kiểm tra phiên bản của các công cụ để đảm bảo môi trường đã được cấu hình chính xác.

3.5.2. Đăng ký và lấy API Key từ OpenWeather

Để sử dụng dữ liệu thời tiết, cần đăng ký tài khoản trên OpenWeather và tạo API Key. Sau khi xác thực tài khoản và đăng nhập, API Key được cấp sẽ được lưu trong tệp cấu hình môi trường (.env) nhằm phục vụ quá trình gọi API. Đồng thời, tệp chứa khóa API cần được thêm vào danh sách loại trừ của Git để tránh làm lộ thông tin khi đưa mã nguồn lên GitHub.

3.5.3. Đưa code lên GitHub

Sau khi hoàn thiện ứng dụng, mã nguồn được quản lý thông qua Git và lưu trữ trên GitHub. Quy trình bao gồm khởi tạo Git repository, liên kết với repository từ xa trên GitHub, thực hiện commit các thay đổi và đẩy mã nguồn lên nhánh chính. Việc sử dụng GitHub giúp quản lý phiên bản hiệu quả, đồng thời hỗ trợ tích hợp tự động với Vercel trong quá trình triển khai ứng dụng.

```
PS D:\weather-app> git add .
PS D:\weather-app> git commit -m "update backend"
[main f0a7c9e] update backend
 1 file changed, 3 insertions(+), 1 deletion(-)
PS D:\weather-app> git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 20 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 596 bytes | 596.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Phuc1306cs/weather-app.git
 1a67e68..f0a7c9e  main -> main
PS D:\weather-app>
```

Hình 14. bash git đưa code lên git

3.5.4. Triển khai lên Vercel

Có hai cách triển khai lên Vercel:

- Cách 1 - Sử dụng Vercel Dashboard: Đăng nhập tại vercel.com bằng tài khoản GitHub. Nhấn "New Project" và chọn repository weather-app từ GitHub. Vercel tự động nhận diện là ứng dụng web tĩnh, không cần cấu hình build. Trước khi deploy, thêm biến môi trường: vào Settings > Environment Variables, thêm WEATHER_API_KEY với giá trị API key. Nhấn "Deploy", ứng dụng sẽ được deploy với URL dạng weather-app-username.vercel.app.
- Cách 2 - Sử dụng Vercel CLI từ terminal Ubuntu: Chạy lệnh `vercel login` để đăng nhập. Trong thư mục dự án, chạy `vercel` để khởi tạo project. Nhập các thông tin cấu hình theo hướng dẫn. Chạy `vercel --prod` để deploy lên production. Thêm biến môi trường bằng lệnh: `vercel env add WEATHER_API_KEY`.

```

1a67e68..f0a7c9e main -> main
PS D:\weather-app> npx vercel
Vercel CLI 54.20.0 (Node.js 22.23.1)
Inspect https://vercel.com/phuc-s-projects8/weather-app/Ewg4XRPM1Xw9rC6rQSDbkLMF1X8E
Preview https://weather-cuev8sg71-phuc-s-projects8.vercel.app

✓ Ready in 11s
To deploy to production (phuc-weather.vercel.app), run `vercel --prod`
PS D:\weather-app> npx vercel --prod
Vercel CLI 54.20.0 (Node.js 22.23.1)
Inspect https://vercel.com/phuc-s-projects8/weather-app/9uR1qiHqJmpd2DawL7Cy8r6ZNhbm
Production https://weather-lvp5155uo-phuc-s-projects8.vercel.app
Aliased https://phuc-weather.vercel.app

✓ Ready in 12s
PS D:\weather-app>

```

Hình 15. bash deploy source lên vercel

3.5.5. Bảo mật API Key với Environment Variables

Một vấn đề quan trọng khi xây dựng ứng dụng web tích hợp API là bảo mật API key. Nếu API key được nhúng trực tiếp vào file JavaScript front-end, bất kỳ ai cũng có thể xem và lạm dụng.

Giải pháp cho ứng dụng tĩnh: Đối với ứng dụng HTML/CSS/JS thuần, API key sẽ hiển thị trong source code của trình duyệt. Để giảm thiểu rủi ro, có thể áp dụng các biện pháp: sử dụng Vercel Functions để tạo một proxy API endpoint ẩn API key phía server, giới hạn API key chỉ cho phép gọi từ domain cụ thể (OpenWeather hỗ trợ tính năng này), và đặt giới hạn rate limiting cho tài khoản.

Phương án tối ưu nhất là tạo một Vercel Function (serverless function) đóng vai trò proxy: người dùng gọi endpoint của Vercel Function, Function đọc API key từ Environment Variable của Vercel, sau đó Function gọi OpenWeather API và trả kết quả về cho client. Theo cách này, API key không bao giờ xuất hiện ở phía client.

3.6. Nâng cấp chức năng lưu vị trí với Supabase

Sau khi hoàn thành các chức năng chính như tra cứu thời tiết, hiển thị bản đồ và dự báo 5 ngày, hệ thống được nâng cấp thêm chức năng lưu vị trí yêu thích. Chức năng này giúp người dùng lưu lại các địa điểm đã tra cứu để có thể xem lại nhanh chóng trong những lần sử dụng sau.

Để triển khai chức năng này, hệ thống sử dụng Supabase làm cơ sở dữ liệu lưu trữ. Front-end không kết nối trực tiếp đến Supabase mà gửi yêu cầu đến các Serverless Functions trên Vercel. Các API này sẽ xử lý dữ liệu và thực hiện thao tác lưu, lấy danh sách hoặc xóa vị trí trong cơ sở dữ liệu. Cách triển khai này giúp hệ thống có kiến trúc rõ ràng hơn và hạn chế việc để lộ thông tin kết nối database ở phía trình duyệt.

3.6.1. Thiết kế bảng `saved_locations`

Bảng `saved_locations` được sử dụng để lưu trữ các vị trí yêu thích của người dùng. Mỗi bản ghi trong bảng tương ứng với một vị trí đã được lưu. Các thông tin chính được lưu bao gồm mã định danh người dùng theo trình duyệt, tên thành phố, quốc gia, tọa độ, nhiệt độ, độ ẩm và thời gian cập nhật.

Các trường chính của bảng gồm:

Bảng 4. Các trường dữ liệu chính lưu trên Supabase

Tên trường	Ý nghĩa
<code>id</code>	Mã định danh duy nhất của bản ghi
<code>user_id</code>	Mã định danh người dùng
<code>location_key</code>	Khóa nhận diện vị trí, dùng để tránh lưu trùng
<code>city</code>	Tên thành phố
<code>country</code>	Mã quốc gia
<code>display_address</code>	Tên vị trí hiển thị trên giao diện
<code>latitude</code>	Vĩ độ của vị trí
<code>longitude</code>	Kinh độ của vị trí
<code>temperature</code>	Nhiệt độ tại thời điểm lưu

humidity	Độ ẩm tại thời điểm lưu
save_count	Số lần vị trí được lưu lại
created_at	Thời gian tạo bản ghi
updated_at	Thời gian cập nhật bản ghi

Bảng được thiết kế có ràng buộc không cho lưu trùng cùng một vị trí đối với cùng một client_id. Nếu người dùng lưu lại một vị trí đã tồn tại, hệ thống sẽ cập nhật bản ghi cũ thay vì tạo thêm bản ghi mới. Điều này giúp dữ liệu gọn gàng và dễ quản lý hơn.

3.6.2. Cơ chế user_id

Khi người dùng lưu vị trí, xem danh sách vị trí đã lưu hoặc xóa vị trí, user_id sẽ được gửi kèm đến Serverless Function. Nhờ đó, hệ thống có thể lấy đúng danh sách vị trí tương ứng với user hiện tại.

Cơ chế này phù hợp với phạm vi đề tài vì đơn giản, dễ triển khai và yêu cầu xây dựng hệ thống đăng nhập.

3.6.3. API lưu vị trí

API lưu vị trí được xây dựng trong file api/save-location.js. Khi người dùng bấm nút “Lưu vị trí”, hệ thống lấy dữ liệu thời tiết hiện tại đang hiển thị trên giao diện, bao gồm tên thành phố, quốc gia, tọa độ, nhiệt độ và độ ẩm.

Sau đó, Front-end gửi dữ liệu này đến API /api/save-location bằng phương thức POST. Serverless Function nhận dữ liệu, kiểm tra thông tin đầu vào, tạo location_key cho vị trí và kết nối đến Supabase để lưu dữ liệu vào bảng saved_locations.

Nếu vị trí chưa tồn tại, hệ thống thêm một bản ghi mới. Nếu vị trí đã tồn tại với cùng client_id và location_key, hệ thống cập nhật bản ghi cũ và tăng số lần lưu. Sau khi lưu thành công, giao diện hiển thị thông báo xác nhận cho người dùng và tải lại danh sách vị trí đã lưu.

3.6.4. API lấy danh sách và xóa vị trí đã lưu

API lấy danh sách và xóa vị trí được xây dựng trong file api/saved-locations.js. Với phương thức GET, API nhận client_id từ trình duyệt, sau đó truy vấn bảng saved_locations trên Supabase để lấy danh sách các vị trí đã lưu tương ứng. Danh sách được sắp xếp theo thời gian cập nhật mới nhất để người dùng dễ dàng xem các vị trí vừa lưu.

Với phương thức DELETE, API nhận client_id và id của vị trí cần xóa. Serverless Function sẽ kiểm tra và xóa bản ghi tương ứng trong Supabase. Sau khi xóa thành công, giao diện sẽ tải lại danh sách vị trí đã lưu để cập nhật kết quả mới nhất.

3.7. Kiểm thử chức năng (Testing Plan)

Để đảm bảo ứng dụng hoạt động đúng và ổn định trên môi trường production, nhóm thực hiện kế hoạch kiểm thử theo các bước:

Bảng 5. Kiểm thử các chức năng

STT	Test Case	Input	Expected Output	Kết quả
TC01	Tìm kiếm thành phố hợp lệ	"Ho Chi Minh"	Hiển thị thời tiết TPHCM	Pass
TC02	Tìm kiếm thành phố tiếng Việt	"Hà Nội"	Hiển thị thời tiết Hà Nội	Pass
TC03	Tìm kiếm thành phố nước ngoài	"Tokyo"	Hiển thị thời tiết Tokyo	Pass
TC04	Nhập thành phố không tồn tại	"abcxyz123"	Hiển thị thông báo lỗi	Pass
TC05	Ô tìm kiếm trống	(empty)	Không gọi API, có thông báo nhắc nhở	Pass
TC06	Lưu vị trí thời tiết	Đã lưu thành công	Lưu thời tiết trong Supabase	Pass
TC07	Xóa vị trí thời tiết đã lưu	Vị trí đã lưu bị xóa	Vị trí lưu bị xóa trong Supabase	Pass
TC08	Đăng nhập	Đăng nhập vào user	Đăng nhập thành công	Pass
TC09	Đăng xuất	Đăng xuất khỏi user	Bạn đã đăng xuất	Pass
TC10	Đăng kí	Lưu user vào database	Bạn đã đăng kí thành công	Pass

CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Môi trường thực nghiệm

Quá trình thực nghiệm được thực hiện nhằm kiểm tra khả năng hoạt động của ứng dụng Weather App sau khi hoàn thành các chức năng chính và triển khai lên nền tảng Vercel. Ứng dụng được xây dựng bằng HTML, CSS và JavaScript ở phía Front-end, kết hợp với Vercel Serverless Functions để xử lý dữ liệu phía server.

Các dịch vụ và công cụ được sử dụng trong quá trình thực nghiệm bao gồm:

Bảng 6. Dịch vụ và công cụ được sử dụng trong quá trình thực nghiệm

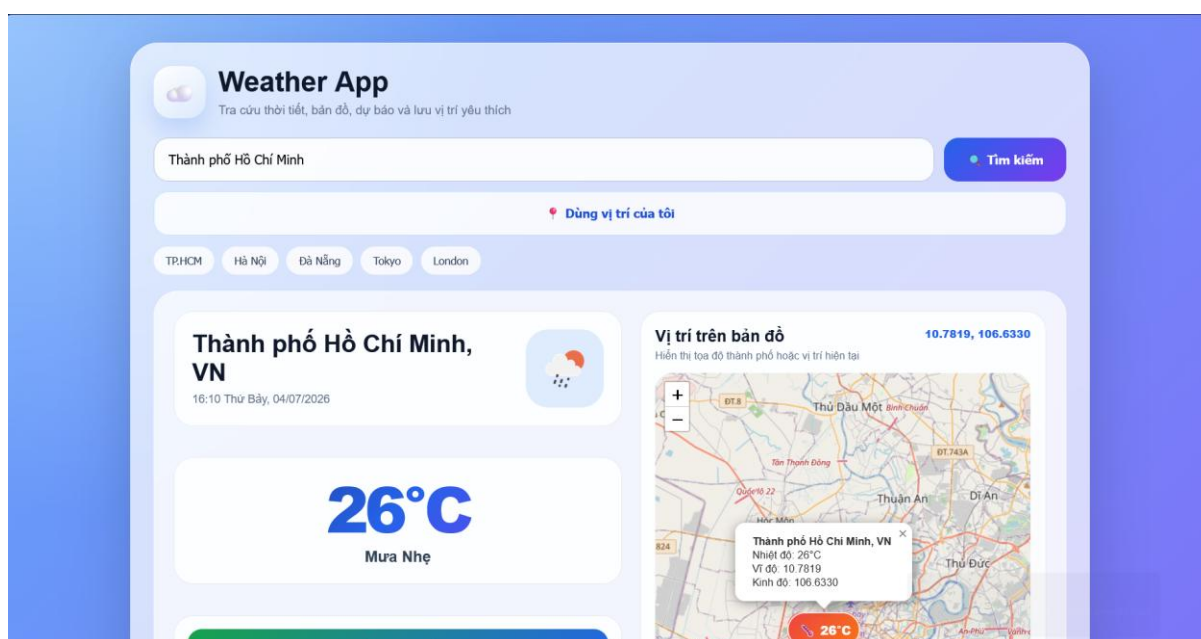
Thành phần	Công cụ/Dịch vụ sử dụng
Hệ điều hành phát triển	Ubuntu/Windows
Trình soạn thảo mã nguồn	Visual Studio Code
Ngôn ngữ lập trình	HTML, CSS, JavaScript
API thời tiết	OpenWeather API
Bản đồ	Leaflet và OpenStreetMap
Nền tảng triển khai	Vercel
Quản lý mã nguồn	Git và GitHub
Cơ sở dữ liệu	Supabase
Môi trường chạy API	Vercel Serverless Functions

Trong quá trình kiểm thử, ứng dụng được chạy thử ở môi trường cục bộ bằng lệnh `vercel dev`, sau đó được triển khai lên môi trường thực tế thông qua Vercel. Các chức năng được kiểm tra bao gồm tra cứu thời tiết, sử dụng vị trí hiện tại, hiển thị bản đồ, dự báo 5 ngày, lưu vị trí yêu thích và quản lý danh sách vị trí đã lưu.

4.2. Kết quả triển khai giao diện ứng dụng

Giao diện của ứng dụng được thiết kế theo hướng đơn giản, trực quan và dễ sử dụng. Người dùng có thể nhập tên thành phố vào ô tìm kiếm hoặc sử dụng nút định vị để lấy thông tin thời tiết tại vị trí hiện tại. Ngoài ra, ứng dụng còn cung cấp các nút chọn nhanh một số thành phố phổ biến nhằm giúp thao tác tra cứu thuận tiện hơn.

Bố cục giao diện được chia thành các khu vực chính gồm: khu vực tìm kiếm, khu vực hiển thị thông tin thời tiết hiện tại, khu vực bản đồ, khu vực dự báo thời tiết 5 ngày và khu vực danh sách vị trí đã lưu. Cách bố trí này giúp người dùng dễ dàng quan sát các thông tin quan trọng trên cùng một giao diện.



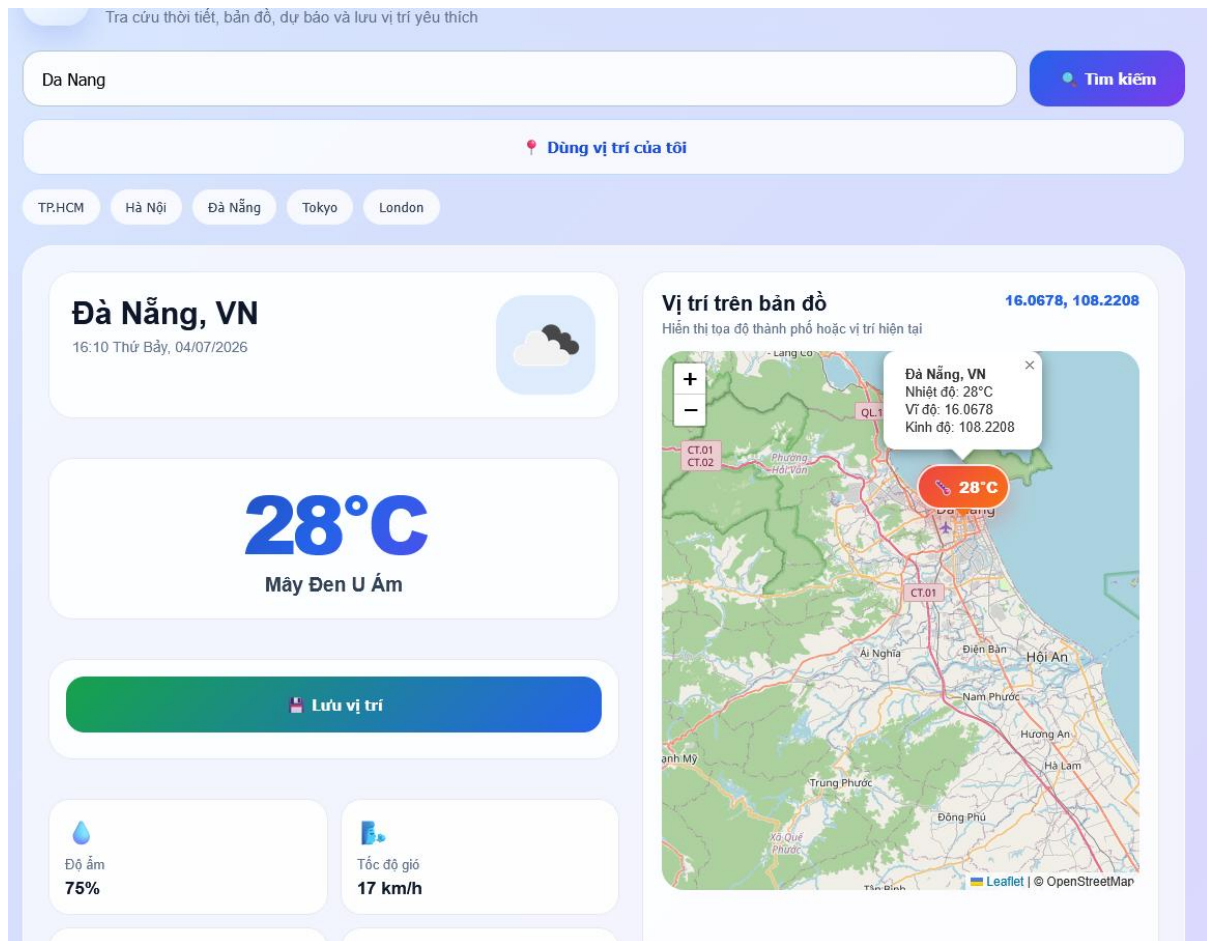
Hình 16. Giao diện chính của ứng dụng Weather App

4.3. Kết quả chức năng tra cứu thời tiết

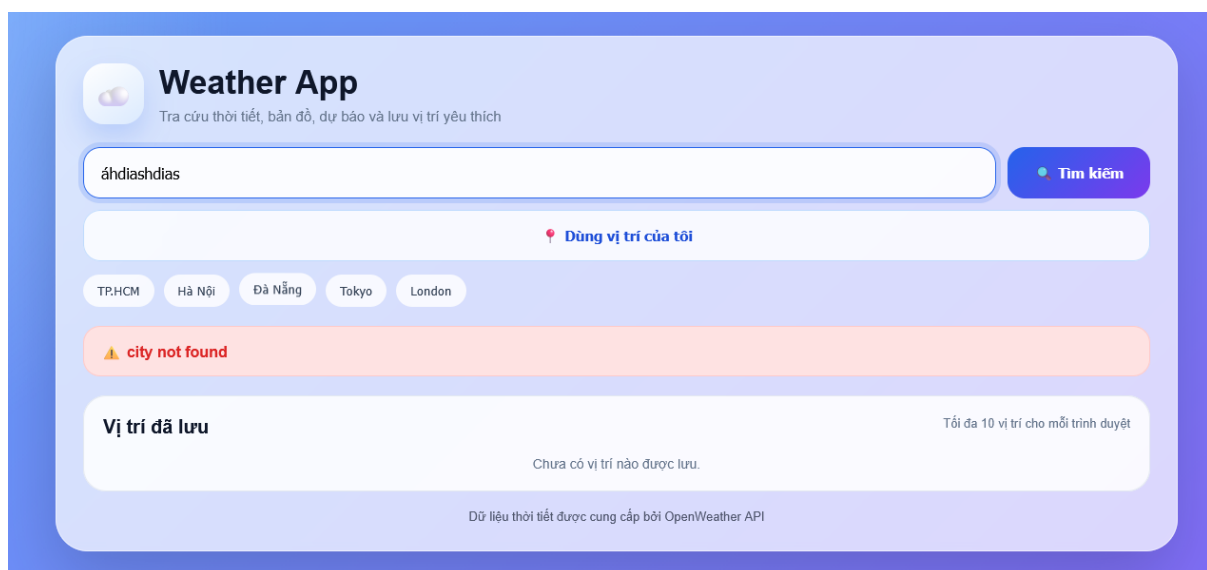
Chức năng tra cứu thời tiết cho phép người dùng nhập tên thành phố để lấy thông tin thời tiết hiện tại. Khi người dùng gửi yêu cầu, Front-end sẽ gọi đến API nội bộ `/api/weather`. API này tiếp tục gửi request đến OpenWeather API, nhận dữ liệu phản hồi và trả kết quả về cho trình duyệt.

Các thông tin thời tiết được hiển thị bao gồm tên thành phố, quốc gia, thời gian, nhiệt độ hiện tại, mô tả thời tiết, độ ẩm, tốc độ gió, nhiệt độ cảm nhận, áp suất khí quyển, tầm nhìn, độ che phủ mây, thời gian mặt trời mọc và mặt trời lặn.

Kết quả thực nghiệm cho thấy chức năng tra cứu hoạt động đúng với các thành phố hợp lệ như Thành phố Hồ Chí Minh, Hà Nội, Đà Nẵng, Tokyo hoặc London. Khi nhập sai tên thành phố, hệ thống hiển thị thông báo lỗi để người dùng biết và nhập lại.



Hình 17. Kết quả tra cứu thời tiết theo tên thành phố

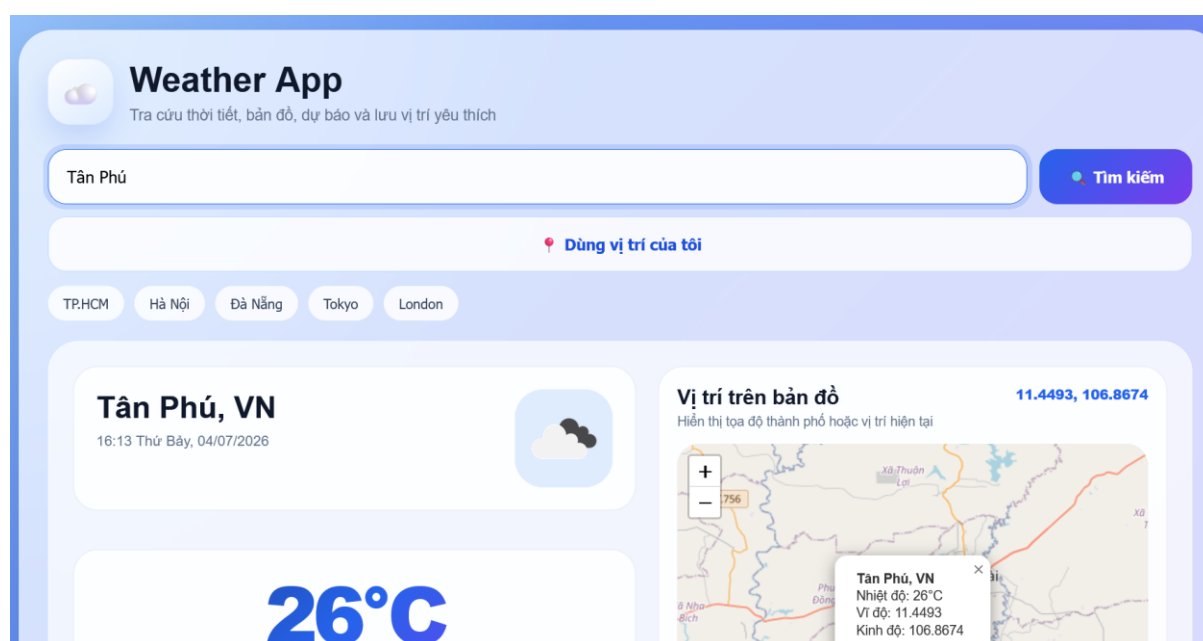


Hình 18. Thông báo lỗi khi nhập thành phố không hợp lệ

4.4. Kết quả chức năng sử dụng vị trí hiện tại

Ứng dụng hỗ trợ chức năng lấy thời tiết theo vị trí hiện tại của người dùng thông qua Geolocation API của trình duyệt. Khi người dùng bấm nút “Dùng vị trí của tôi”, trình duyệt sẽ yêu cầu quyền truy cập vị trí. Nếu người dùng cho phép, hệ thống lấy tọa độ vĩ độ và kinh độ, sau đó gửi đến API /api/weather để lấy dữ liệu thời tiết tương ứng.

Kết quả thực nghiệm cho thấy chức năng định vị hoạt động đúng khi trình duyệt được cấp quyền vị trí. Trong trường hợp người dùng từ chối quyền định vị hoặc trình duyệt không hỗ trợ chức năng này, ứng dụng hiển thị thông báo lỗi phù hợp.



Hình 19. Chức năng vị trí của tôi

4.5. Kết quả hiển thị bản đồ vị trí

Ứng dụng tích hợp thư viện Leaflet để hiển thị bản đồ vị trí của thành phố hoặc tọa độ hiện tại của người dùng. Dữ liệu tọa độ được lấy từ phản hồi của OpenWeather API, sau đó truyền vào bản đồ để hiển thị marker tại vị trí tương ứng.

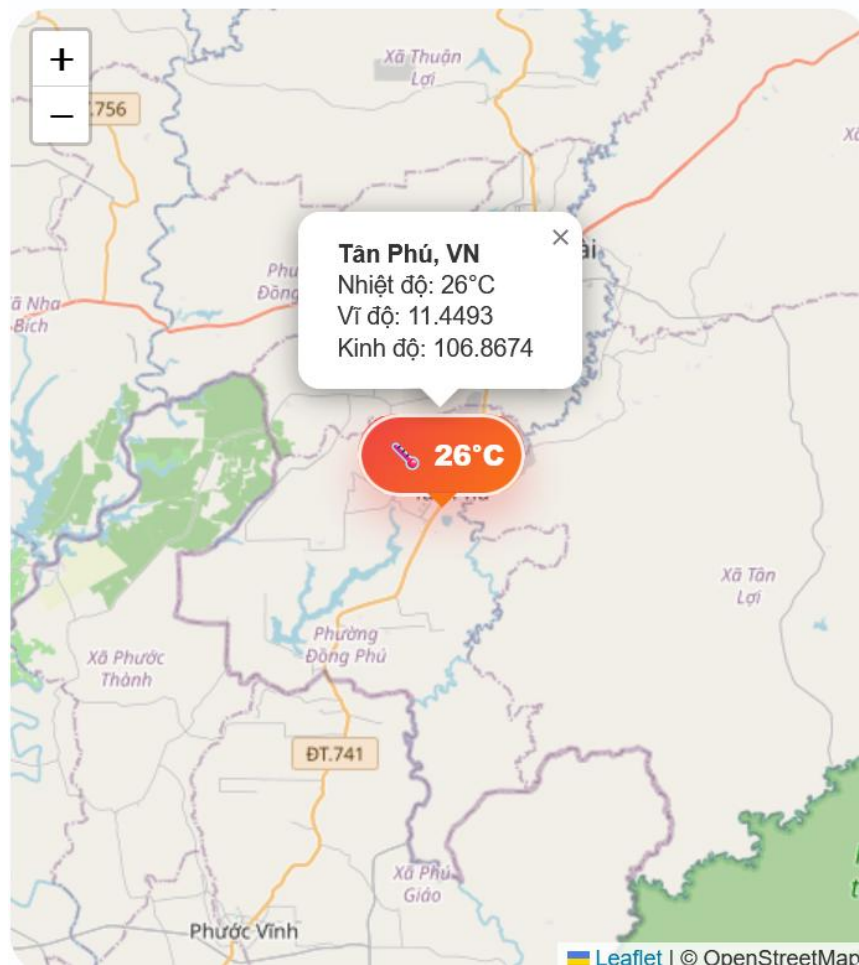
Trong phiên bản nâng cấp, marker trên bản đồ có thể hiển thị thêm nhiệt độ hiện tại của vị trí đang tra cứu. Điều này giúp người dùng vừa quan sát được vị trí địa lý, vừa nhanh chóng nắm được thông tin nhiệt độ ngay trên bản đồ.

Kết quả thực nghiệm cho thấy bản đồ hiển thị đúng tọa độ của địa điểm được tra cứu. Marker được cập nhật khi người dùng tìm kiếm một thành phố mới hoặc sử dụng vị trí hiện tại.

Vị trí trên bản đồ

11.4493, 106.8674

Hiển thị tọa độ thành phố hoặc vị trí hiện tại



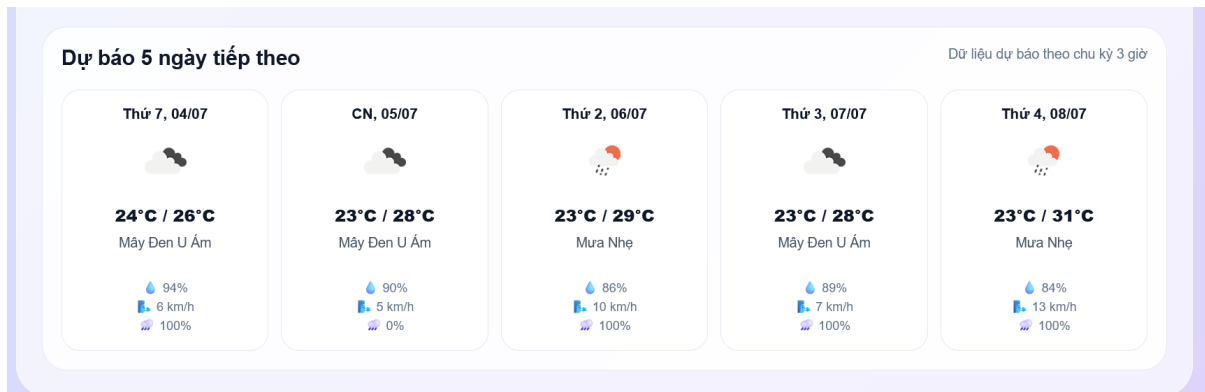
Hình 20. Bản đồ hiển thị vị trí thành phố và marker hiển thị

4.6. Kết quả chức năng dự báo thời tiết 5 ngày

Ngoài chức năng hiển thị thời tiết hiện tại, ứng dụng còn được mở rộng thêm chức năng dự báo thời tiết 5 ngày tiếp theo. Dữ liệu dự báo được lấy từ OpenWeather Forecast API thông qua Serverless Function /api/forecast.

Do dữ liệu dự báo của OpenWeather được cung cấp theo chu kỳ 3 giờ, hệ thống xử lý và nhóm dữ liệu theo từng ngày trước khi hiển thị lên giao diện. Mỗi thẻ dự báo hiển thị các thông tin cơ bản như ngày dự báo, biểu tượng thời tiết, nhiệt độ thấp nhất, nhiệt độ cao nhất, độ ẩm, tốc độ gió và khả năng mưa.

Kết quả thực nghiệm cho thấy dữ liệu dự báo được hiển thị đầy đủ, giúp người dùng có thể xem nhanh xu hướng thời tiết trong những ngày tiếp theo.



Hình 21. Giao diện dự báo thời tiết 5 ngày

4.7. Kết quả nâng cấp chức năng lưu vị trí của user với Supabase

Sau khi hoàn thành các chức năng chính, hệ thống được nâng cấp thêm chức năng lưu vị trí yêu thích bằng Supabase. Đây là phần nâng cấp đã được triển khai trong hệ thống nhằm giúp ứng dụng có khả năng lưu trữ dữ liệu người dùng ở mức cơ bản.

Khi người dùng đăng kí và sau đó đăng nhập vào trang web thì người dùng mới có chức năng lưu vị trí. Người dùng tra cứu thời tiết thành công, nút “Lưu vị trí” cho phép lưu lại vị trí hiện tại vào cơ sở dữ liệu Supabase. Dữ liệu được lưu bao gồm tên thành phố, quốc gia, tọa độ, nhiệt độ, độ ẩm và thời gian cập nhật.

Sau khi lưu thành công, hệ thống hiển thị thông báo xác nhận và cập nhật lại danh sách vị trí đã lưu trên giao diện. Người dùng có thể xem lại các vị trí này mà không cần nhập lại tên thành phố.

Weather App
Tra cứu thời tiết, bản đồ, dự báo và lưu vị trí yêu thích

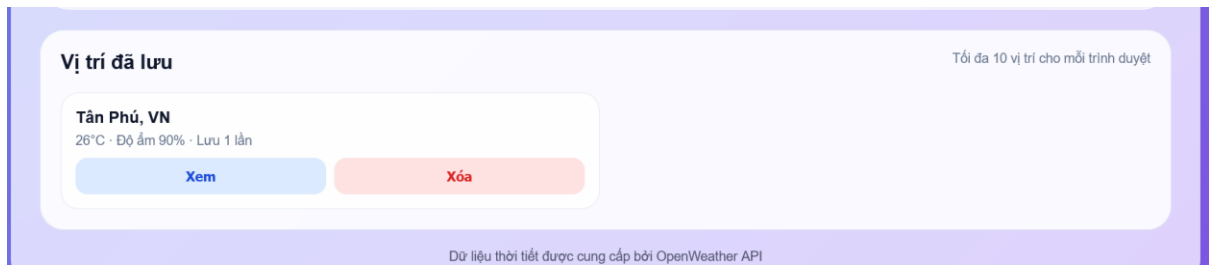
Tài khoản người dùng
Đã đăng nhập: **thong** Đăng xuất

☒ Đăng ký và đăng nhập thành công.

Thành phố Hồ Chí Minh Tìm kiếm

📍 Dừng vị trí của tôi

Hình 22. Đăng kí và đăng nhập thành công



Hình 23. Lưu vị trí trên giao diện

4.8. Kết quả lưu dữ liệu vào Supabase

Dữ liệu vị trí sau khi lưu được ghi vào bảng `saved_locations` trên Supabase. Mỗi bản ghi đại diện cho một vị trí đã được người dùng lưu lại. Các thông tin được lưu gồm `user_id`, tên thành phố, quốc gia, tọa độ, nhiệt độ, độ ẩm, số lần lưu và thời gian cập nhật.

Kết quả thực nghiệm cho thấy khi người dùng bấm nút “Lưu vị trí”, dữ liệu được gửi từ Front-end đến Vercel Serverless Function, sau đó được lưu thành công vào Supabase. Điều này chứng minh hệ thống đã kết nối được giữa giao diện người dùng, API phía server và cơ sở dữ liệu.

id	client_id	location_key	city	country
2088217d-bb03-41ef-9e0e-d650f2f1ef	aec5e6fa-6530-4f8e-8ef2-5a6f56b65	tokyo-jp	Tokyo	JP
2f7f16a1-2500-4411-a308-4fe0fedf48e	1854d48c-d4c7-49df-a30e-686a986f	thong-tay-hoi-vn	Thong Tay Hoi	VN
6879f4a0-28e5-4209-b33c-83e9e7d5	63103f42-0413-4f82-b006-c074b421c	a-nang-vn	Đà Nẵng	VN
9beae871-ce0d-4f11-a50c-7046727f6	57429533-9394-4151-9b8b-e7f7924e	tan-phu-vn	Tân Phú	VN
9ddb47f1-6c8b-4ad6-b2e6-7216bac4f	aec5e6fa-6530-4f8e-8ef2-5a6f56b65	thanh-pho-ho-chi-minh-vn	Thành phố Hồ Chí Minh	VN
a8891587-bd62-41d8-8348-98e9430e	a2f7343e-4dd2-4a70-997c-7852e053	thanh-pho-ho-chi-minh-vn	Thành phố Hồ Chí Minh	VN

Hình 24. Bảng `saved_locations` trên Supabase

id	username	password_hash	created_at
4a15e101-7729-4c97-8f5d-037a9ff195	thong	\$2b\$10\$d9l0pe7WbDage2OcQCcZteH	2026-07-14 00:31:21.909005
d58117c0-2136-4fad-a73d-4f8fd0c6b8	phuchaha	\$2b\$10\$Q7rYbGc/8FTQKozz78A1w.tAz	2026-07-08 15:32:42.243281
fe32034d-445c-4443-bac4-d73bea67	phuc	\$2b\$10\$EmvBIADHSkqCnd5EYC.a.fIR	2026-07-08 14:29:55.03610e

Hình 25. Bảng `user` được đăng kí và lưu vào Supabase

CHƯƠNG 5: KẾT LUẬN

5.1. Kết quả đạt được

Sau quá trình tìm hiểu, thiết kế và triển khai, đề tài đã xây dựng được một ứng dụng Web tra cứu thời tiết cơ bản, có khả năng hoạt động trực tuyến thông qua nền tảng Vercel. Ứng dụng cho phép người dùng nhập tên thành phố để xem thông tin thời tiết hiện tại như nhiệt độ, độ ẩm, tốc độ gió, áp suất khí quyển, mô tả thời tiết và thời gian mặt trời mọc, mặt trời lặn.

Bên cạnh chức năng tra cứu thời tiết, hệ thống còn tích hợp bản đồ Leaflet để hiển thị vị trí địa lý của thành phố được tìm kiếm. Điều này giúp người dùng dễ hình dung vị trí của khu vực đang xem thời tiết. Ngoài ra, ứng dụng còn có chức năng dự báo thời tiết 5 ngày, giúp người dùng theo dõi xu hướng thời tiết trong những ngày tiếp theo.

Một điểm nâng cấp của đề tài là tích hợp Supabase để lưu vị trí yêu thích. Người dùng có thể lưu lại các địa điểm đã tra cứu, xem lại danh sách vị trí đã lưu và xóa vị trí khi không còn cần thiết. Chức năng này giúp ứng dụng có tính thực tế hơn, không chỉ dừng lại ở việc hiển thị dữ liệu thời tiết tạm thời.

Ngoài ra, đề tài cũng đã triển khai được mô hình sử dụng Vercel Serverless Functions để xử lý các request đến OpenWeather API và Supabase. Việc này giúp hạn chế việc để lộ API Key ở phía Front-end, đồng thời làm cho kiến trúc hệ thống rõ ràng và dễ mở rộng hơn.

5.2. Ưu điểm của hệ thống

Ứng dụng có giao diện đơn giản, dễ sử dụng và phù hợp với người dùng phổ thông. Các chức năng chính như tìm kiếm thời tiết, sử dụng vị trí hiện tại, hiển thị bản đồ, xem dự báo và lưu vị trí được bố trí rõ ràng trên giao diện.

Hệ thống sử dụng các công nghệ phổ biến như HTML, CSS, JavaScript, OpenWeather API, Leaflet, Vercel và Supabase. Đây là các công nghệ phù hợp với phạm vi của đề tài, dễ triển khai và có khả năng mở rộng trong tương lai.

Việc triển khai ứng dụng lên Vercel giúp website có thể truy cập trực tuyến thông qua Internet. Đồng thời, việc sử dụng Environment Variables giúp bảo vệ các thông tin quan trọng như API Key và khóa kết nối Supabase.

5.3. Hạn chế của hệ thống

Mặc dù đã hoàn thành các chức năng chính, hệ thống vẫn còn một số hạn chế. Ứng dụng chưa có chức năng đăng nhập tài khoản người dùng, vì vậy dữ liệu vị trí yêu thích hiện

tại được phân biệt thông qua `client_id` lưu trên trình duyệt. Nếu người dùng đổi thiết bị, đổi trình duyệt hoặc xóa dữ liệu trình duyệt thì danh sách vị trí đã lưu có thể không còn được hiển thị.

Ngoài ra, dữ liệu thời tiết phụ thuộc vào OpenWeather API nên khi API bị lỗi, hết giới hạn request hoặc mất kết nối Internet, ứng dụng sẽ không thể lấy dữ liệu mới. Chức năng bản đồ hiện tại mới chỉ hiển thị vị trí và marker cơ bản, chưa có các lớp dữ liệu nâng cao như bản đồ mưa, mây hoặc nhiệt độ theo khu vực.

Ứng dụng cũng chưa có hệ thống quản trị, chưa có phân quyền người dùng và chưa lưu lịch sử tra cứu chi tiết. Đây là những điểm có thể tiếp tục cải tiến nếu phát triển hệ thống ở quy mô lớn hơn.

5.4. Hướng phát triển trong tương lai

Trong tương lai, hệ thống có thể được phát triển thêm chức năng đăng ký và đăng nhập tài khoản người dùng. Khi đó, danh sách vị trí yêu thích sẽ được đồng bộ theo tài khoản, giúp người dùng có thể truy cập dữ liệu trên nhiều thiết bị khác nhau.

Ngoài ra, ứng dụng có thể bổ sung chức năng lưu lịch sử tra cứu thời tiết, cảnh báo thời tiết xấu, hiển thị biểu đồ nhiệt độ, lượng mưa và độ ẩm theo thời gian. Phần bản đồ cũng có thể được nâng cấp bằng cách tích hợp thêm các lớp bản đồ thời tiết như mây, mưa, gió hoặc nhiệt độ.

Về mặt kỹ thuật, hệ thống có thể được cải tiến thêm về bảo mật, tối ưu tốc độ tải trang, kiểm soát lỗi API tốt hơn và triển khai thêm Progressive Web App để người dùng có thể cài đặt ứng dụng trên điện thoại hoặc máy tính.

5.5. Kết luận chung

Nhìn chung, đề tài đã hoàn thành mục tiêu xây dựng một ứng dụng Web tra cứu thời tiết có giao diện trực quan, dễ sử dụng và có khả năng triển khai thực tế trên nền tảng Cloud. Ứng dụng không chỉ lấy và hiển thị dữ liệu thời tiết từ OpenWeather API mà còn tích hợp bản đồ, dự báo 5 ngày và chức năng lưu vị trí yêu thích bằng Supabase.

Thông qua quá trình thực hiện đề tài, nhóm đã hiểu rõ hơn về cách xây dựng một ứng dụng Web sử dụng API, cách triển khai lên Vercel, cách bảo mật API Key bằng Environment Variables và cách kết nối cơ sở dữ liệu Cloud bằng Supabase. Đây là nền tảng quan trọng để tiếp tục phát triển các ứng dụng Web hoàn chỉnh hơn trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] Vercel, “Deploying to Vercel,” Vercel Documentation. Link truy cập: “<https://vercel.com/docs/deployments#vercel-rest-api>”.
- [2] Vercel, “Accessing Deployments through Generated URLs,” Vercel Documentation. Link truy cập: “<https://vercel.com/docs/deployments/generated-urls#url-components>”.
- [3] Vercel, “Environments,” Vercel Documentation. Link truy cập: “<https://vercel.com/docs/deployments/environments>”.
- [4] Viblo, “[DevOps Thực Chiến] Vercel từ A-Z: Phép thuật Deploy ‘Chạm là chạy’,” Viblo. [Trực tuyến]. Link truy cập: “<https://viblo.asia/p/devops-thuc-chien-vercel-tu-a-z-phep-thuat-deploy-cham-la-chay-XP4WEo6pL7G>”.
- [5] YouTube, “Cách Triển Khai Website Frontend Lên Vercel,” YouTube. Link truy cập: “<https://www.youtube.com/watch?v=M9C1I-k3KuQ&t=1s>”.
- [6] OpenWeather, “One Call API,” OpenWeather API Documentation. Link truy cập: “<https://openweathermap.org/api/one-call-api#how>. [Truy cập: 24/06/2026]”.

Link Demo: <https://youtu.be/d9anjiaCp-g>